Topologically Restricted P-Computing: Embedding Large Sparse Networks onto Scalable Hardware

Szymon Ablewicz

CRANN, Trinity College Dublin, Ireland

Supervised by Prof. Stefano Sanvito

October 10, 2026

Probabilistic computers and quantum annealers solve hard optimization and combinatorial problems with networks of coupled spins, and on regular fixed hardware probabilistic computers could one day reach billions of p-bits. The networks of real problems, however, tend to exhibit greater, less local and more irregular connectivity than such hardware, so in practice networks must be minor-embedded onto it. Large sparse networks have no embedding method that is both compact and guaranteed, so the field falls back on the complete graph embedding, blind to sparsity, or it compromises on FPGAs, which do not scale like fixed hardware. This minor-embedding problem lies at the forefront of the field for real-world application of probabilistic spin logic technology and my project tries to tackle the problem directly, on the King's graph topology. I improved PSSA, the best known heuristic for this hardware, and extended it with border constraints (PSSA wBC) so that a large network can be partitioned into small clusters, each on its own patch with its border nodes on the perimeter, leaving only the cut edges between them to route. My Divide and Conquer method routes the cut edges by heuristic search, carrying the classic strategy of electronic design automation onto fixed hardware, and as a highly undeveloped proof of concept it already embeds random cubic graphs with 1,000 vertices in approximately 3.4 times less area than the complete graph embedding. My fallback method instead routes the cut edges through a complete graph embedding of the border nodes alone, so it is guaranteed, runs in linear time and scales to large problems. The fallback embedding's size is set by the number of border nodes, and I found that minimizing border nodes in graph partitioning is opposed to the classic goal of minimizing the cut, so with this method, embedding compactly becomes a question of partitioning. At a million vertices, the fallback method needs 2.76 times less area than the complete graph embedding on random cubic graphs and 4.43 times less on Erdős–Rényi graphs, and on both families its gain grows with the size of the network. It already wins on expanders, seemingly the hardest sparse graphs, which is a good sign that it could become a general embedder for sparse networks that is sensitive to sparsity and other graph properties, unlike any other scalable method in the field.

Contents

1 Introduction and theory **2**
- 1.1 Highly connected networks on sparse hardware 2
- 1.2 Minor-embedding . . . 2
- 1.3 From the square lattice to the King's graph . . . 2
- 1.4 Guaranteed embeddings and heuristics . . . 3
- 1.5 Graph families of interest . . . 3
- 1.6 My work . . . 4

2 PSSA and PSSA wBC **5**
- 2.1 Probabilistic Swap-Shift Annealing (PSSA) . . . 5
  - 2.1.1 Algorithm 1: the annealing search . . . 5
  - 2.1.2 Algorithm 2: the terminal search . . . 8
  - 2.1.3 The 2021 improvements and performance 8
- 2.2 Implementations and code availability . . . 9
- 2.3 Improvements to PSSA . . . 10
  - 2.3.1 Gibbs-weighted swap and shift moves . 10
  - 2.3.2 Computing ΔE locally . . . 11
  - 2.3.3 Geometric annealing . . . 12
  - 2.3.4 Translation to C++ . . . 12
- 2.4 PSSA with border constraints . . . 12
  - 2.4.1 Border-aware initial placement . . . 13
  - 2.4.2 Mode 1: fixed border nodes . . . 13
  - 2.4.3 Mode 2: free border nodes . . . 13
- 2.5 Results and limits . . . 14

3 Divide and Conquer method **15**
- 3.1 Min-cut partitioning . . . 16
- 3.2 Placing and routing the clusters . . . 16
- 3.3 Results and limits . . . 17

4 Fallback method **18**
- 4.1 The construction . . . 19
- 4.2 The size of the embedding . . . 20
- 4.3 Graph partitioning to minimize border node count 20
  - 4.3.1 Mt-KaHyPar . . . 21
  - 4.3.2 GGP + T-FM . . . 22
  - 4.3.3 Comparison of the two partitioners . . 22
- 4.4 Results . . . 22

5 Discussion **23**
- 5.1 Divide and Conquer vs the fallback method . . 23
- 5.2 The treewidth law and the constant coefficient . 23
- 5.3 Where each method is at its best . . . 24
- 5.4 What limits each method . . . 24

6 Further research **25**
- 6.1 The fallback method as a general sparse embedder 25
- 6.2 Larger clusters with Divide and Conquer . . . 26
- 6.3 Divide and Conquer at scale . . . 26
- 6.4 Running on real hardware . . . 26

7 Conclusion **27**

1 Introduction and theory

1.1 Highly connected networks on sparse hardware

Probabilistic computers and quantum annealers solve hard optimization and combinatorial problems with networks of spins coupled through ferromagnetic and antiferromagnetic interactions, which are gradually annealed into the ground states of their energy landscapes (Aadit et al. 2022; Choi 2008). In a probabilistic computer each spin is a p-bit, a binary stochastic number generator whose probabilities are set by its inputs. P-bits are built from CMOS circuits or from stochastic magnetic tunnel junctions (sMTJs), and unlike quantum computers they work at room temperature (Kaiser and Datta 2021; Chowdhury et al. 2023). In fact, networks of p-bits can even emulate the thermodynamics of quantum systems free of the notorious sign problem (Camsari et al. 2019). As my project proposal set out (Sanvito 2026), a key problem in implementing networks carrying logic on a probabilistic computer is that the p-bit networks emulating a given problem are usually highly connected. This is a serious limitation on a dedicated chip, where the connections between the p-bits are determined by hardware constraints, which in practice give a regular and fixed architecture with nearest-neighbour connections. Yet it is exactly such regular, fixed hardware that can grow to the scale probabilistic computing aims for. The inherent sparsity and locality of these fixed hardware topologies is a strength, as with graph colouring, every p-bit of one colour is updated at once, so the number of flips per second grows linearly with the number of p-bits (Aadit et al. 2022) and in this way the low chromatic number of these hardware topologies leads to great parallelism in the annealing process. In contrast, highly connected or all-to-all connected hardware, which avoids the connectivity problem altogether, lacks sparsity and locality and thus provides little parallelism, and furthermore, it must wire every p-bit to every other, so that the number of connections grows quadratically with the logical nodes of the network, which can't be sustained at large scales (Aadit et al. 2022; Nikhar et al. 2024). With sMTJs, a single chip of a million p-bits is projected to be feasible, as magnetic memory technology has already integrated billions of magnetic tunnel junctions into CMOS-compatible chips (Nikhar et al. 2024), and so it is on regular, fixed hardware that probabilistic computers could one day reach billions of p-bits.

The field has so far largely sidestepped the problem. Aadit et al. (2022), of Camsari's group at the University of California, Santa Barbara, at the forefront of probabilistic computing, found that typical minor-embedding algorithms often fail to find a mapping for their problems, or need far too many hardware spins when they do, and so the group sparsifies its networks and runs them on arbitrary connectivity FPGAs, whose connections can be configured to match the network itself, so that no embedding is needed at all. FPGAs, however, do not scale like fixed hardware, as a digital p-bit takes up so much of an FPGA that even a high-end device holds only tens of thousands of them, against millions on a single core with sMTJs (Chowdhury et al. 2023), and so the group's million p-bit computer already spans 18 FPGAs (Aadit et al. 2026). In this report I instead tackle the problem directly, by improving how large sparse networks are minor-embedded onto regular, fixed hardware, here the King's graph (Section 1.3), so that they fit compactly and reliably at the scale p-computing is heading for.

1.2 Minor-embedding

The standard way to run a network on hardware with sparser connectivity is minor-embedding (Choi 2008). The problem network is the input graph I and the chip is the hardware graph H, in which each hardware vertex is a spin, a p-bit in probabilistic computing or a qubit in quantum annealing. Every vertex i of the input graph becomes a super vertex $\phi(i)$, a connected chain of hardware vertices that act as one logical spin, and every edge of the input graph is realized by at least one hardware edge between the two super vertices it joins. An assignment of super vertices to all the input vertices is called a super vertex placement ϕ, and when it satisfies all of this it is a minor-embedding, as I is then a minor of H, ie. I can be obtained from H through the three basic operations of deleting vertices, deleting edges and contracting edges (Diestel 2005, Section 1.7), and Section 2.1 states these conditions formally. Finding a minor-embedding is NP-hard in general (Cai et al. 2014), which is what calls for heuristic embedding algorithms to push for compact embeddings.

Physically, the network is a set of spins $m_i = \pm 1$ with energy

$$E(\mathbf{m}) \;=\; -\sum_{i<j} J_{ij}\, m_i m_j \;-\; \sum_i h_i\, m_i,$$

whose couplings J_{ij} may take either sign, ferromagnetic or antiferromagnetic, as set by the problem. The couplings inside a super vertex are strictly ferromagnetic, $J > 0$, and strong enough that its spins agree in the ground states, so that the embedded network keeps the ground states of the original, and setting their strength is the parameter setting problem of Choi (2008). I noticed that three fields split a vertex into a chain in exactly the way described, each under its own name. In graph theory, splitting a vertex into a connected chain is the inverse of contracting an edge, which is how a minor is defined, and super vertices are known there as branch sets (Diestel 2005, Section 1.7). In probabilistic computing the same split is the COPY gate method, which divides one p-bit into two p-bits coupled ferromagnetically, and which Aadit et al. (2022) call sparsification, with fusion as its inverse, proving that it leaves the ground states unchanged. Chowdhury et al. (2023) describe this sparsification as "similar in spirit" to minor-embedding, "even though the objective here is to not find an embedding but to sparsify an existing graph". However, I find that sparsifying a network with COPY gates and minor-embedding it are one and the same operation with two different aims, and a super vertex is exactly a chain of COPY gates laid out on hardware. A network sparsified in this way thus keeps the original as a minor, so every minor-embedding of the sparsified network is also one of the original.

1.3 From the square lattice to the King's graph

I started this project with focus on the square lattice, where every spin is connected to its four nearest neighbours other than at the boundary of the chip, and where minor-embedding itself began, as Choi (2011) recounts, with the square lattice hardware of Kaminsky et al. The square lattice, however, is planar, and a graph is a minor of a large enough grid exactly when it is planar (Diestel 2005, Exercise 4.13). Robertson et al. (1994) made "large enough" explicit, as every planar graph G is a minor of the square grid of side $2(|V(G)| + 2|E(G)|)$. The square lattice thus hosts every planar network and nothing else. By the theorem of Kuratowski and Wagner, a graph is planar if

and only if it has neither K_5 nor $K_{3,3}$ as a minor (Diestel 2005, Theorem 4.4.6), so the square lattice cannot host even K_5, the complete graph on five vertices, let alone the highly connected networks of real problems. I realized that standard sparsification cannot help either, as splitting vertices with COPY gates only ever produces networks that have the original as a minor, so no number of COPY gates can make a non-planar network planar, and the obstruction lies in the hardware itself. Whether a network fits on a square lattice at all is thus decided in linear time by a planarity test (Hopcroft and Tarjan 1974), with no search at all, yet I find that heuristic searches have been used for it instead, as when Aadit et al. (2022) found that embedding their factorizer onto a square grid with minorminer "always seems to fail". Regardless, I concluded that my choice of hardware itself must therefore allow crossings, and as a consequence it must be non-planar, so I moved to the King's graph. The King's graph adds the two crossing diagonals to every face of the square lattice, connecting every spin to its eight neighbours with the same local pattern everywhere, so that a chip is built simply by tiling it, and with only four colours it keeps the parallelism of a sparse, local network (Aadit et al. 2022). It is the hardware graph of Hitachi's CMOS annealer, which has 102,400 vertices (Sugie et al. 2021), while D-Wave's quantum annealers use their own sparse hardware graphs, Chimera, Pegasus and Zephyr (Macaskill-Smith et al. 2026).

The King's graph is a universal hardware architecture. Any graph on n vertices is a minor of $\mathrm{KG}_{n-1,n-1}$, through the complete graph embedding of Okuyama et al. (2016), which embeds the complete graph K_n and thus every graph on n vertices, in a closed-form pattern with no search at all (Fig. 1). It is the operation of Section 1.2 taken to its extreme as every vertex is split into a chain long enough to meet every other, just as in the TRIAD construction of Choi (2011) before it. This complete graph embedding is provably near optimal, as $\mathrm{KG}_{L,L}$ holds the complete graph on $L+1$ vertices and no $\mathrm{KG}_{L,L}$ can hold one on more than $2L$, a bound which was later tightened to $L+2$, so that it is optimal to within a single vertex (Sugie et al. 2021). Nor could any other hardware of fixed degree do much better, as none can hold a complete graph on more than of order $\sqrt{|V(H)|}$ vertices, so the King's graph itself loses at most a constant factor to any other relevant topology (Sugie et al. 2021). This one complete graph embedding pattern runs through the whole report, as PSSA starts from it (Section 2.1) and the fallback method turns it into its routing structure (Section 4).

Throughout this report I measure an embedding by the side L of the smallest King's graph $\mathrm{KG}_{L,L}$ it fits on, and by its area L^2, ie. the number of hardware vertices of that chip, against the $(n-1)^2$ hardware vertices of the complete graph embedding.

1.4 Guaranteed embeddings and heuristics

There are two kinds of embedding methods, guaranteed minor-embedding patterns and heuristic minor-embedding algorithms. Guaranteed patterns embed a whole graph family in closed form, with no search at all, and Sugie et al. (2021) list three of them, for complete graphs, complete bipartite graphs and Cartesian products of graphs (Choi 2011; Okuyama et al. 2016; Zaribafiyan et al. 2017), while planar graphs, though not embedded by a closed-form pattern, are guaranteed to fit on a large enough square lattice (Section 1.3), and in fact any planar network is embedded in $O(n^2 \log n)$ time by the algorithm of Tamassia (1987). The patterns for complete graphs, namely the TRIAD construction and the so-called best known complete graph embedding, I reconstructed myself (Fig. 1). All of them scale to any size, but each only really fits the family it was built for. As for the complete graph embedding in particular, though it's the only pattern that fits every graph, it's blind to sparsity as it uses the same $(n-1)^2$ hardware vertices for every graph on n vertices, however sparse the network graph (Section 4). In contrast, heuristic algorithms search for an embedding, with PSSA (Sugie et al. 2018; Sugie et al. 2021) the best known heuristic on the King's graph (Section 2.1) and minorminer (Cai et al. 2014) the standard on D-Wave's hardware. They give compact embeddings, but each is a search on an NP-hard problem with no guarantee of success, and the improved PSSA of Sugie et al. (2021) has been demonstrated on random cubic graphs of up to about 1,000 vertices, on the largest chip they tested, $\mathrm{KG}_{320,320}$ with 102,400 vertices, which is still toy problem size next to the million p-bits that probabilistic computers already reach (Section 1.1). Critically in regards to these heuristic algorithms, only their successes can be trusted, as a returned embedding can be checked directly, whereas a failure is ambiguous as it cannot tell you whether no embedding exists, whether the network must first be sparsified further, or whether the search simply ran out of time, and such failures are common in practice, as Camsari's group found (Section 1.1).

1.5 Graph families of interest

From what I have found, other than the trivial case of two-dimensional lattices, all the networks of the problems of interest in probabilistic computing and quantum annealing come down to four graph families: complete graphs, complete bipartite graphs, sparse structured bipartite graphs and sparse expanders. Two-dimensional lattices, such as that of the two-dimensional Ising model, are already solved, as they are themselves part of the square lattice, and so of the King's graph (Section 1.3), yet these in practice cover only a marginal part of the problems of interest. As for the first two families, dense networks don't scale in practice, as their number of parameters grows as n^2 making the problem intractable at scale, and so in practice they are sparsified with the aim of retaining their structure and logic, ie. the ground states and general shape of their energy landscape, and it is the sparsified network that is then run on the hardware. A complete bipartite network can be pruned into a sparse structured bipartite one. In practice this is a common approach where a restricted Boltzmann machine is pruned so that it can be simulated tractably, and a sparse network can in fact match its accuracy with a hundred times fewer parameters (Niazi et al. 2024). A similar approach can be taken for complete networks, which, with the aim of preserving their logic, can become sparse expanders, ie. sparse graphs that stay well connected throughout. Expanders are the most notorious family for minor-embedding, as even their best embedding needs a side L that grows linearly with n, just like the complete graph embedding (Section 3.3). The way I see it, a method that embeds expanders well is the most universally applicable embedding strategy, as all other sparse graphs are seemingly easier to embed, and unsurprisingly, random d-regular and Erdős–Rényi graphs, which are expanders, are the standard benchmark graph families of minor-embedding (Sugie et al. 2021; Macaskill-Smith et al. 2026). I also found them to recur among the hard problems of probabilistic computing and quantum annealing, as some of the field's biggest known problems are natively sparse and

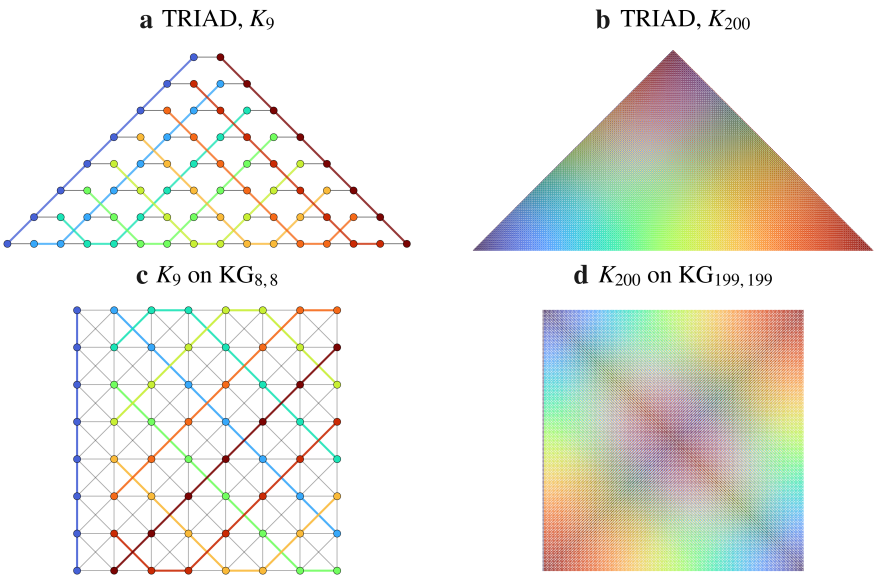


Figure 1 | The two guaranteed complete graph embeddings, in my reconstructions, for K_9 (left) and K_{200} (right), with each super vertex in its own colour. **a, b**, The TRIAD construction of Choi (2011), which lays out K_n as a triangle of chains of $n-1$ vertices each, where each black line realizes one logical edge of K_n. **c, d**, The best known complete graph embedding of Okuyama et al. (2016), as described by Hung and Roychowdhury (2024), which embeds K_n on $KG_{n-1,n-1}$, with vertex 1 along the first column and every other vertex a chain running diagonally across the square, where the grey lines are the available hardware edges between different super vertices.

Graph family	Example problem	Best known embedding	This project
Complete	travelling salesman	complete graph embedding (Sugie et al. 2021)	–
Complete bipartite	restricted Boltzmann machines	complete bipartite component of the TRIAD (Choi 2011)	–
Sparse structured bipartite	sparse deep Boltzmann machines (Niazi et al. 2024), 3D Edwards–Anderson spin glasses (Aadit et al. 2026)	complete bipartite component of the TRIAD, and PSSA up to about 1,000 vertices	Divide and Conquer (Sections 3 and 6.3)
Sparse expanders	Max-Cut on random sparse graphs (Ye 2003), random Boolean satisfiability (Aadit et al. 2022; Aadit et al. 2026)	complete graph embedding, and PSSA up to about 1,000 vertices (Sugie et al. 2021)	the fallback method (Section 4)

Table 1 | The four graph families that the problems of interest come down to, and how each is embedded on the King's graph.

well connected, such as Max-Cut, the textbook Ising problem, whose standard benchmark library, the G-set, includes many random sparse graphs (Ye 2003), and Boolean satisfiability, the classic NP-complete problem, benchmarked on random instances near its phase transition (Aadit et al. 2022; Aadit et al. 2026). Random cubic graphs, ie. random 3-regular graphs, are the running benchmark of this report, and Erdős–Rényi graphs are also tested. Nevertheless, regardless of significance and feasibility, the two dense families are already solved by the closed-form embeddings of Section 1.4, the complete graph embedding even provably near optimal (Section 1.3), and this project comes down to tackling the open problem of the two sparse families, as Table 1 sums up.

For large sparse networks, then, nothing is both compact and guaranteed, as the heuristics reach only about 1,000 vertices and none of the guaranteed patterns was built for them, so the only real scalable solution in the field as of now is to fall back to the complete graph embedding, at a huge overhead in hardware vertices, and it is this gap that this report sets out to close.

1.6 My work

My work builds on the PSSA algorithm, the best known heuristic for the King's graph (Section 1.4), which I worked to improve from its publicly available reconstructions (Hung and Roychowdhury 2024; Macaskill-Smith et al. 2026) and then extended to PSSA with border constraints (PSSA wBC), which can hold chosen vertices on the perimeter of an embedding (Sections 2.1 and 2.4). It is with PSSA wBC that I embed networks far larger than PSSA can on its own. A large network is partitioned into clusters small enough for PSSA, and every cluster is embedded on its own patch of the King's graph, with its border nodes, ie. its vertices with edges to other clusters, held on the perimeter of the patch, where they can be reached. What remains is then to connect the clusters, by realizing every cut edge between them through the available hardware between the patches, and I found two ways of doing so. The first is my Divide and Conquer method (Section 3), which follows the classic strategy of electronic design automation for laying out circuits on chips, by partitioning the network so that as few edges as possible are cut, placing the patches in a set layout and then searching for a route for every cut edge, a simpler problem than minor-embedding, as the placement of the clusters is fixed from the start. Divide and Conquer, in spirit, is the same strategy that Camsari's group used just recently in June 2026 to map a 3D spin glass network across the 18 FPGAs of their million p-bit computer (Aadit et al. 2026), and my method takes it onto fixed hardware, where every cut edge must be routed through the chip itself. It is compact, needing a median of 3.4 times less area than the complete graph embedding on random cubic graphs with 1,000 vertices, but it still tackles an NP-hard problem and

fails to scale to large networks. It is therefore best seen as a heuristic minor-embedding algorithm in its own right, most promising on sparse structured bipartite networks that exhibit community structure and whose clusters connect locally, such as perhaps 3D spin glasses (Section 6.3). The second method of mine is the fallback method (Section 4), which needs no routing search at all, as it connects the clusters through a complete graph embedding of their border nodes alone, so that every border node can meet every other, and thus cut edge routing is guaranteed and scales to networks of any size, in linear time. Its size is set by the number of border nodes rather than of cut edges, and as I found that minimizing one is in fact opposed to minimizing the other, it calls for partitions that minimize the border nodes, so that a better embedding becomes a matter of a better partition. On expanders, seemingly the hardest of the sparse graphs, its gain over the complete graph embedding grows with the size of the network, which is a good sign that it could become a general way of embedding sparse networks.

Section 2.1 describes standalone PSSA and PSSA with border constraints, Sections 3 and 4 the two methods and their results, Section 5 compares the methods, and Section 6 sets out plans for future research.

2 PSSA and PSSA wBC

From benchmarks in quantum annealing, the field analogous to probabilistic computing, the general consensus is that the D-Wave heuristic embedding algorithm minorminer (Cai et al. 2014) is the single best embedding algorithm, with the highest success rate of the heuristics benchmarked by Macaskill-Smith et al. (2026). Although this is the case for the D-Wave topologies Chimera, Pegasus and Zephyr, minorminer's superiority doesn't quite hold for the King's graph. According to Ohno and Togawa (2024), minorminer is more unreliable when embedding onto the King's graph lattice, and the Probabilistic Swap-Shift Annealing (PSSA) algorithm was speculated to be superior on this hardware topology.

PSSA is a heuristic minor-embedding algorithm specifically built for the King's graph, as it uses the complete graph embedding pattern of Okuyama et al. (2016) in its construction, and was developed by Yuki Yoshida as a submission to the "Hokkaido University & Hitachi 2nd New-Concept Computing Contest 2017". The algorithm won the contest, and further work was then done by Yoshida and the contest organizers to produce Sugie et al. (2018), in which the philosophy of the approach and the procedure were outlined with pseudocode and the performance of the algorithm was benchmarked against minorminer on three canonical graph families, random cubic graphs and random connected graphs with 20% and 50% edge density, and the follow-up paper Sugie et al. (2021), where PSSA was further developed and benchmarked against its original script. The 2018 benchmarks showed PSSA to successfully embed more nodes than minorminer's algorithm for a given search time on a given hardware size. It embedded random cubic graphs with up to 50% more vertices, and at 20% and 50% edge density it was the only algorithm to embed graphs with more than 53 vertices, so that on the King's graph it did not lose on any of the tested families. For the bigger picture, on the Chimera graph of D-Wave's annealers PSSA tied with minorminer's algorithm on random cubic graphs, did slightly worse at 20% edge density and better at 50% (Sugie et al. 2018). These two papers encompass all of the developments on the PSSA algorithm until now, and in fact there exist only a handful of papers that mention the algorithm on any level, of which the others will be discussed in Section 2.2. The philosophy and procedure of the up-to-date PSSA method will now be outlined using the original 2018 pseudocode along with the 2021 pseudocode, which outlines the creators' improvements on the method.

2.1 Probabilistic Swap-Shift Annealing (PSSA)

PSSA is specified across the two papers by two pseudocodes. Algorithm 1, given in Sugie et al. (2018) and reproduced with minor notational changes in Sugie et al. (2021), constitutes the core simulated annealing search itself. In fact, PSSA anneals a label on every hardware vertex, the input vertex whose super vertex it belongs to, so the NP-hard problem of minor-embedding is itself solved by the same kind of annealing a p-computer runs, only over multi-state variables rather than p-bits. Algorithm 2, which is introduced in the 2021 paper, is a successive phase which calls Algorithm 1 at its line 1 and then applies a deterministic cleanup, called the *terminal search*, to whatever placement comes back. Its primary purpose is to rescue runs in which Algorithm 1 has failed to find a correct minor-embedding, though when Algorithm 1 has already succeeded the same procedure serves to trim redundant hardware vertices from the embedding to reduce the number of used hardware vertices and save chip space.

2.1.1 Algorithm 1: the annealing search

Search space and cost function. Recall from Section 1.2 that a super vertex placement is a mapping

$$\phi : V(I) \to 2^{V(H)} \tag{1}$$

which assigns to every vertex $i \in V(I)$ of the input graph a subset $\phi(i) \subset V(H)$ of hardware vertices called the super vertex of i. A placement ϕ therefore specifies the images of all input vertices simultaneously, and so constitutes one candidate placement on the hardware. The notation $2^{V(H)}$ is the power set of $V(H)$ denoting the set of all subsets of hardware vertices so that each $\phi(i)$ is a subset of the hardware vertices. For a placement ϕ to be a minor-embedding it must satisfy these three conditions:

$$\text{(M1)} \quad \forall i \in V(I) : \ \phi(i) \neq \emptyset \text{ and } \phi(i) \text{ induces a connected subgraph of } H, \tag{2}$$

$$\text{(M2)} \quad \forall i, j \in V(I),\ i \neq j : \ \phi(i) \cap \phi(j) = \emptyset, \tag{3}$$

$$\text{(M3)} \quad \forall (i, j) \in E(I) : \ \exists (u, v) \in \phi(i) \times \phi(j) \text{ with } (u, v) \in E(H). \tag{4}$$

These conditions are what make the placement a correct representation of the input graph. The first condition (M1) requires each super vertex to be non-empty and connected, so that the hardware spins comprising it can be forced to act as a single logical spin. The second condition (M2) requires the super vertices to be pairwise disjoint, so that no hardware node is claimed by two logical spins at once. Condition (M3) is what makes the minor-embedding faithful to the input graph as it imposes that each edge of the input graph must be realized by at least one hardware edge running between the corresponding super vertices, where here an edge is represented by a nearest-neighbour adjacency. The placement ϕ is thus a minor embedding precisely when all three hold.

Algorithm 1: PSSA (Sugie et al. 2018)

Input : Input graph I and hardware graph H
Output : Super vertex placement ϕ_{best}
Ensure : $|V(I)| \leq |V(H)|$, $|E(I)| \leq |E(H)|$
Require : Function $E_{\text{emb}}(\phi)$, Eq. (6), $t_{\max}$, schedule, guiding_pattern

```
// prepare initial placement of super vertices
```
$\phi \leftarrow$ guiding_pattern divided into $|V(I)|$ super vertices
$\phi_{\text{best}} \leftarrow \phi$; **if** $E_{\text{emb}}(\phi_{\text{best}}) = |E(I)|$ **return** ϕ_{best} and terminate `// minor found`
```
// improve super vertex placement through simulated annealing
```
for $t = 0$ **to** $t_{\max}$ **do**
  move $\leftarrow$ swap or shift, randomly selected according to schedule
  if *move is swap* **then** `// see Fig. 2b`
    $i, k \leftarrow (i, k) \in E(I)$, randomly selected
    $j \leftarrow j \in V(I)$ with $\phi(j)$ neighbouring $\phi(k)$ in H, randomly selected
    $\phi_{\text{proposed}} \leftarrow \phi$ with $\phi(i)$ and $\phi(j)$ swapped
  else if *move is shift* **then** `// see Fig. 2c, d`
    $i, u \leftarrow i \in V(I)$ with $|\phi(i)| > 1$ and $u \in \text{Leaf}[\phi(i)]$, both randomly selected
    allow_any_direction_shift $\leftarrow$ true or false according to schedule
    if *allow_any_direction_shift* **then** `// see Fig. 2d`
      $j, v \leftarrow j \in V(I)$, $v \in \text{Leaf}[\phi(j)]$ with v adjacent to u in H, randomly selected
    else `// see Fig. 2c`
      $j, v \leftarrow j \in V(I)$, $v \in \text{Leaf}[\phi(j)]$ with v adjacent to u along guiding_pattern, randomly selected
    $\phi_{\text{proposed}} \leftarrow \phi$ with u deleted from $\phi(i)$ and assigned to $\phi(j)$
  ```
  // evaluate acceptance of proposed move
  ```
  $\Delta E \leftarrow E_{\text{emb}}(\phi_{\text{proposed}}) - E_{\text{emb}}(\phi)$; $T(t) \leftarrow$ temperature according to schedule
  if $\exp(\Delta E / T(t)) >$ *random_float* $\in [0, 1)$ **then**
    ```
    // accept and update
    ```
    $\phi \leftarrow \phi_{\text{proposed}}$
    if $E_{\text{emb}}(\phi_{\text{best}}) < E_{\text{emb}}(\phi)$ **then**
      $\phi_{\text{best}} \leftarrow \phi$
      if $E_{\text{emb}}(\phi_{\text{best}}) = |E(I)|$ **then return** ϕ_{best} and terminate `// minor found`
return ϕ_{best} `// even if` $E_{\text{emb}}(\phi_{\text{best}}) < |E(I)|$`, ie. minor not found`

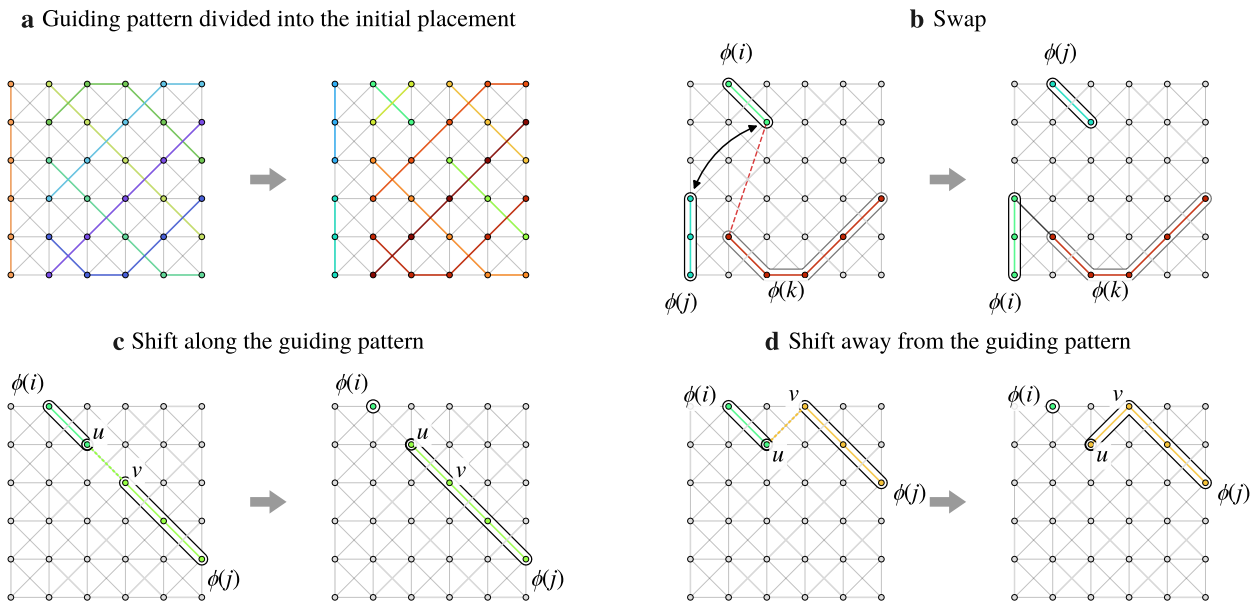


Figure 2 | The initial placement and the moves of PSSA on a King's graph $\text{KG}_{6,6}$, redrawn after Fig. 2 of Sugie et al. (2018). Each panel goes from before (left) to after (right). **a**, The guiding pattern, the complete graph embedding of K_7 with one colour per super vertex (left), divided into the super vertices of an Erdős–Rényi input graph with 10 vertices and 15 edges (right). **b**, A swap, with only the super vertices involved in colour: the input edge (i, k) is not yet embedded (red dashed), so $\phi(i)$ and $\phi(j)$ exchange places, where $\phi(j)$ is adjacent to $\phi(k)$. Afterwards $\phi(i)$ is next to $\phi(k)$ and the edge is embedded (black). **c, d**, The two kinds of shift, on the same placement, with only $\phi(i)$ and $\phi(j)$ in colour: the leaf u is deleted from $\phi(i)$ and attached to $\phi(j)$ at its leaf v (dashed). **c**, A shift along the guiding pattern: u and v lie on the same diagonal chain of the guiding pattern, so $\phi(j)$ grows along that chain. **d**, A shift away from the guiding pattern: the same leaf u is attached instead to a super vertex on a different chain, and $\phi(j)$ bends across the chain boundary.

Minor-embedding is NP-hard (Cai et al. 2014), thus, identifying the ϕ that satisfy all three conditions is an NP-hard problem. Therefore, in practice PSSA decides to relax the conditions, and does not search the set of minor-embeddings directly but instead it searches a wider configuration space that satisfies the first two conditions (M1) and (M2), but not necessarily (M3),

$$\Phi = \left\{\phi : V(I) \to 2^{V(H)} \text{ satisfying (M1) and (M2)}\right\} \tag{5}$$

so that Φ is the set of all admissible layouts on the hardware. Condition (M3) is then treated not as a constraint defining Φ but as an objective to be maximized, measured by the number of input edges actually represented on the hardware,

$$E_{\text{emb}}(\phi) := \Big|\big\{(i,j) \in E(I) \;\big|\; \exists u \in \phi(i),\, v \in \phi(j) \text{ with } (u,v) \in E(H)\big\}\Big|, \tag{6}$$

for which $E_{\text{emb}}(\phi) \le |E(I)|$ always, with

$$\begin{aligned} E_{\text{emb}}(\phi) = |E(I)| &\iff \phi \text{ satisfies (M3)} \\ &\iff \phi \text{ is a minor embedding.} \end{aligned} \tag{7}$$

The search therefore ends successfully the moment the objective reaches its maximum possible value, which is the condition checked at lines 2 and 22. If the iteration budget $t_{\max}$ runs out first, the best placement found is returned anyway (line 23) and nothing can be concluded as to whether an embedding exists at all. Throughout the annealing phase each $\phi(i)$ stays a path in H, as the initial pieces are paths and both moves preserve that. The endpoints of these paths are written $\text{Leaf}[\phi(i)]$, and it is on these leaves that the Swap and Shift moves outlined below act.

Initial placement. At line 1, the initial placement for super vertices is built from a *guiding pattern* which on a King's graph $\text{KG}_{L,L}$ is the best known complete graph embedding of the K_{L+1} complete graph due to Okuyama et al. (2016). A complete graph K_{L+1} is the graph on $L+1$ vertices with all-to-all connectivity. Given this guiding pattern, the $L+1$ super vertices are divided into $|V(I)|$ super vertices of as near equal size as possible, and these super vertices make up ϕ at $t = 0$. See Fig. 2a.

Two things follow from this choice. The first is that when

$$|V(I)| \le L + 1 \tag{8}$$

the division subdivides a placement which already embeds K_{L+1}, so every pair of super vertices stays adjacent and the initial placement is already a minor embedding of any input graph on that many vertices. This is the check at line 2, and critically, it's what guarantees that PSSA never does worse than the complete graph embedding. The second is that when $|V(I)| > L + 1$ the pieces inherit the diagonal orientation of the Okuyama super vertices, so by nature, each one is adjacent to a large number of others on the hardware. Thus, the search begins from a state of fairly high connectivity rather than from a random one.

Moves. New placements are proposed at lines 4–16 by drawing from two move types; the swap and the shift. The two act on different aspects of a placement.

A **swap** (lines 5–8, Fig. 2b) picks an edge $(i,k) \in E(I)$ at random, then picks $j \in V(I)$ such that $\phi(j)$ is adjacent to $\phi(k)$ on H, and exchanges $\phi(i)$ with $\phi(j)$ so that

$$\phi(i) \mapsto \phi(j), \qquad \phi(j) \mapsto \phi(i). \tag{9}$$

Since i is a neighbour of k in the input graph, the effect is to move $\phi(i)$ into a position adjacent to $\phi(k)$, so the proposal is drawn from the unsatisfied connections of $E(I)$ rather than uniformly at random from all connections. This is the sense in which the swap acts directly on the objective.

A **shift** (lines 9–16, Fig. 2c, d) picks a super vertex $\phi(i)$ with $|\phi(i)| > 1$ and one of its leaves $u \in \text{Leaf}[\phi(i)]$, then attaches u to a neighbouring super vertex $\phi(j)$ possessing a leaf $v \in \text{Leaf}[\phi(j)]$ with $(u,v) \in E(H)$, giving

$$\phi(i) \mapsto \phi(i) \setminus \{u\}, \qquad \phi(j) \mapsto \phi(j) \cup \{u\}. \tag{10}$$

The condition $|\phi(i)| > 1$ is what prevents a super vertex from being emptied and (M1) from being violated. Where a swap rearranges super vertices, then, a shift transfers hardware resource between them one vertex at a time, and it is through repeated shifts that the super vertices acquire sizes matched to the input graph rather than the near equal sizes they are given at $t = 0$. One can imagine that vertices with higher degree in the input graph correspond to larger super vertices in the hardware minor-embedding.

The guiding pattern is used a second time here, to classify the shifts. A shift of $u \in \phi(i)$ onto $v \in \phi(j)$ is said to be **along** the guiding pattern if u and v lie in the same Okuyama super vertex, and **away** from it if they lie in different ones. This is the branch at lines 11–15. The distinction matters because the Okuyama super vertices are the diagonal chains which give the initial placement its high connectivity. A shift along the pattern extends a super vertex within one of those chains and preserves the diagonal orientation, whereas a shift away from it moves resource across a chain boundary and degrades that orientation. Restricting to shifts along the pattern therefore keeps the search in the high-connectivity region it starts in, at the cost of reaching only a limited part of Φ, and the schedule described below trades one against the other as the run progresses.

It is worth noting that neither (M1) nor (M2) appears anywhere in Algorithm 1 as an explicit test. Both are guaranteed structurally by how the moves are defined, as a swap exchanges two already disjoint super vertices and so preserves disjointness, while a shift moves a single leaf between super vertices already adjacent on H, and a path remains a path under the deletion of a leaf or the extension from one. Every proposal therefore lands inside Φ by construction, and the algorithm never has to verify satisfaction of the first two conditions, and can focus solely on (M3).

Acceptance. Proposals are accepted by a Metropolis rule at lines 17–18. Writing the change in the objective as

$$\Delta E = E_{\text{emb}}(\phi_{\text{proposed}}) - E_{\text{emb}}(\phi), \tag{11}$$

the proposal is accepted whenever

$$\exp\big(\Delta E / T(t)\big) > r, \qquad r \sim U[0,1), \tag{12}$$

so that any improving move, $\Delta E \ge 0$, is accepted with certainty, while a worsening move is accepted with probability

$$\exp\big(\Delta E / T(t)\big) < 1. \tag{13}$$

The objective here is being maximized, so the $-\Delta E$ familiar from the minimization form of the Metropolis–Hastings algorithm does not appear here. The true Hamiltonian in this case can be thought of as the negative of the $E_{\text{emb}}(\phi)$ function. At large $T(t)$

the exponential is close to unity and almost every proposal gets accepted, so the chain explores Φ freely, whereas as $T(t) \to 0$ only improving moves tend to survive and the search reduces to a greedy algorithm. Accepting worsening moves early is what allows the search to leave local maxima of E_{emb}, according to the standard philosophy behind Metropolis–Hastings, but importantly, because E_{emb} has freedom to decrease throughout the run, the current placement at the end of the run is not necessarily the best one visited. The algorithm therefore keeps ϕ_{best} separately at lines 20–22, updating it only when E_{emb} improves, so that exploration can never degrade the result that is ultimately returned.

Annealing schedules. The object referenced as schedule at lines 4, 11 and 17 controls three quantities, each on a simple linear schedule across two annealing phases of equal length. The first of these is the temperature, given by

$$T(t) = \begin{cases} T_0 \left(1 - \dfrac{2t}{t_{\max}}\right), & 0 \le t < t_{\max}/2, \\ T_{t_{\max}/2} \left(2 - \dfrac{2t}{t_{\max}}\right), & t_{\max}/2 \le t \le t_{\max}, \end{cases} \tag{14}$$

so that each phase begins at a finite temperature and cools to zero. The two-phase form acts as a restart mechanism, as if the first phase cools into a local maximum without producing an embedding, the second raises the temperature back up to $T_{t_{\max}/2}$ and allows the chain to escape it, rather than spending the remaining budget frozen in a configuration it cannot leave.

The move type is then chosen at line 4 by proposing a shift with probability $p_s(t)$ and a swap with probability $1 - p_s(t)$, following the linear schedule

$$p_s(0) = 1, \qquad p_s(t_{\max}) = 0, \tag{15}$$

so that shifts dominate early and swaps late. This orders the two tasks the moves perform, as the super vertices are first grown or shrunk to sizes appropriate to the degrees of the input vertices, and only once those sizes are roughly settled does the search turn to rearranging them so that adjacent input vertices end up placed adjacently on the hardware. Finally, the shift direction is chosen at line 11 by allowing an arbitrary direction with probability $p_a(t)$ and restricting to the guiding pattern with probability $1 - p_a(t)$, where

$$p_a(0) = 0.095, \qquad p_a(t_{\max}) = 0.487. \tag{16}$$

Since p_a remains below $1/2$ throughout, shifts along the guiding pattern are favoured over the whole run and the schedule only widens the search gradually. The reasoning here is that the diagonal orientation of the initial placement is a genuinely good structure and should not be abandoned early, so departures from it are permitted increasingly as the configurations reachable within the pattern are exhausted. The parameter values used by Sugie et al. (2021) are $t_{\max} = 7 \times 10^7$, $T_0 = 60.315$ and $T_{t_{\max}/2} = 33.435$.

Taken as a whole, then, the annealing phase is a Markov chain Monte Carlo method over the configuration space Φ. Each state of the chain is a complete super vertex placement, the chain is initialized at the guiding pattern placement, and each successive state depends only on the current one together with a randomly drawn proposal, with no memory of how the chain arrived there. The transition probabilities carry two layers of randomness, the first being which proposal is drawn, governed by $p_s(t)$ and $p_a(t)$, and the second being whether that proposal is then accepted at all, governed by the Metropolis rule at $T(t)$.

2.1.2 Algorithm 2: the terminal search

Terminal search. A placement returned by Algorithm 1 without an embedding has two characteristic features, and the terminal search is built to exploit both of them. The first is that it occupies the entire hardware, since the initial placement covers every vertex of H and neither move type ever releases one. Some of these assignments are redundant, in that a hardware vertex may belong to $\phi(i)$ without being needed either to keep $\phi(i)$ connected or to realize any edge incident to i. Worse still, such a vertex occupies space that another super vertex might have used to reach a neighbour it is currently failing to reach. The second feature typically arises if the failed run is a near miss, with

$$E_{\text{emb}}(\phi) \lesssim |E(I)| \tag{17}$$

so that only a handful of edges separate the placement from a valid embedding. In this case the algorithm seeks to free the redundant vertices and spend that space on fulfilling the edges still missing, and this is done in two phases. The cleanup phase (lines 2–13) sweeps the hardware vertices repeatedly, deleting $u \in \phi(i)$ whenever is_deletable(u, i) holds. That is, whenever

$$\text{(p1)} \quad \phi(i) \setminus \{u\} \text{ is non-empty and connected, and} \tag{18}$$

$$\text{(p2)} \quad E_{\text{emb}} \text{ is not decreased by the deletion.} \tag{19}$$

The two conditions amount to the requirement that the deletion cost nothing, as (p1) preserves (M1) for $\phi(i)$ while (p2) preserves every edge already embedded. Deleted vertices are collected into the free set

$$U = V(H) \setminus \bigcup_{i \in V(I)} \phi(i) \tag{20}$$

which is empty when the stage begins and grows as the sweep proceeds, terminating once a full pass over the hardware produces no deletion at all. The linking stage (lines 14–16) then iterates over the input edges, and for each $(i, j) \in E(I)$ whose super vertices are not yet adjacent on the hardware, bfs_path(i, j) runs a breadth-first search on the subgraph of H induced by U, looking for a path from $\phi(i)$ to $\phi(j)$. If one is found its vertices are removed from U and absorbed into $\phi(i)$, which now reaches $\phi(j)$ and so embeds the edge, and if none is found then U and ϕ are left unchanged and the next edge is tried. Since the cleanup cannot decrease E_{emb} and the linking stage only ever adds edges, the terminal search never returns a worse placement than the one it was given at the end of Algorithm 1. It does not, however, preserve the path structure of the super vertices, so the restriction to paths applies to the annealing phase alone.

2.1.3 The 2021 improvements and performance

Two further improvements introduced in 2021 appear in neither pseudocode. The first biases the shift proposals by the degrees of the input vertices, so that a leaf is more likely to be transferred towards whichever super vertex is currently under-sized relative to the degree of the vertex it represents. This feature tends to make high degree super vertices large, and low degree ones small. The second addition, and the more consequential one for the work that follows, replaces the linear annealing schedule

Algorithm 2: Improved PSSA with terminal search (Sugie et al. 2021)

Input : Input graph I and hardware graph H
Output : Super vertex placement ϕ
Require : PSSA, is_deletable$(\cdot)$, bfs_path$(\cdot)$

```
// get super vertex placement from PSSA
φ ← PSSA(I, H)
// -- start terminal search --
// create free hardware vertices
Init: U ← ∅                                   // set of free vertices
Init: u ← 0                                   // hardware vertex u ∈ V(H) = {0, ..., |V(H)| − 1}
Init: no_del ← 0                              // vertices scanned since last deletion
while no_del < |V(H)| − 1 do
    // scan hardware vertices u = 0, ..., |V(H)| − 1
    i ← φ^{-1}(u)                             // preimage of u
    if u ∉ U and is_deletable(u, i) then
        // shift u to set of free vertices
        φ(i) ← φ(i) \ {u}
        U ← U ∪ {u}
        no_del ← 0
    else
        no_del ← no_del + 1
    u ← (u + 1) mod |V(H)|                    // check next hardware vertex
// find new super vertex links by breadth-first search on the graph induced by free vertices
for i = 0, ..., |V(I)| − 1 do
    if ∃(i, j) ∈ E(I) with φ(i), φ(j) ⊂ H not connected then
        // search BFS path from φ(i) to φ(j)
        φ, U ← bfs_path(i, j)
return φ                                      // even if E_emb(φ) < |E(I)|, ie. minor not found
```

described above with an exponential one. Within each annealing phase the temperature is updated every 1000 iterations by a cooling factor β, so that

$$T(t) = T_0\,\beta^{\lfloor t/1000 \rfloor} \tag{21}$$

with $\beta = 0.9999$. The factor sits close to unity because the update is applied some $t_{\max}/1000 = 7 \times 10^4$ times over a full run, giving a total decay of $\beta^{70000} \approx 10^{-3}$. The exponential schedule was found to tend to outperform the linear one, on both random cubic and Barabási–Albert graphs. With these in place, the embedding threshold $\bar{V}(L)$, defined as the smallest input size at which the embedding probability falls below 0.95 on $\mathrm{KG}_{L,L}$, was found to scale linearly in L across hardware graphs of up to 102 400 nodes, as

$$\bar{V}(L) \approx \begin{cases} 3.2\,L & \text{(random cubic)}, \\ 2.8\,L & \text{(Barabási–Albert)}, \end{cases} \tag{22}$$

against the baseline $\bar{V}(L) = L + 1$ guaranteed by the complete graph embedding, constituting gains of 42% and 28% respectively over the 2018 version of PSSA.

This, then, is the full philosophy and structure of PSSA as it comes out of the two canonical papers of Sugie et al., and next I turn to the other literature on PSSA and to my own developments of the algorithm.

2.2 Implementations and code availability

The notable papers which discuss PSSA other than the two canonical ones are Hung and Roychowdhury (2024) and the Ember benchmark suite (Macaskill-Smith et al. 2026). In the two canonical papers from the original creators of PSSA, Sugie et al. (2018) and Sugie et al. (2021), the source code of the algorithm is not publicly disclosed, and it's instead displayed in pseudocode in the papers. The Ember and Hung papers prove useful here, as these works reconstructed PSSA from these pseudocodes and published their scripts on GitHub, Ember at https://github.com/zachmacsmith/ember and Hung at https://github.com/jhhung20/King-s-Graph-Mapping. Hung's report is a study of Ising machine designs, part of which maps graphs onto the King's graph with both the complete graph embedding and PSSA, while Ember is a benchmark suite that compares six embedding algorithms, PSSA among them, on D-Wave's hardware graphs over a library of 24,016 graphs.

Although it's worthy of mention that the original source code from the creators is available on the Hitachi contest website, that script lies under a licence which states that the work of the contest winners is the intellectual property of the contestant and the organizers, and I didn't want to risk any legal issues by using it. Due to this restricted access I chose to stick to the publicly available reconstructions from Hung and Ember, though it is worth noting that while the pseudocode implementation works, it was benchmarked against state-of-the-art heuristics in Macaskill-Smith et al. (2026) and was shown to lose clearly to minorminer on D-Wave's hardware graphs, with a success rate of 40% against 61% for minorminer, ranking first only on complete graphs. Thus, although the original PSSA has been demonstrated to beat minorminer on the tested instances in Sugie et al. (2018), that code isn't openly available, and the raw implementations from the pseudocode seem to be inferior, though still in line with most state-of-the-art heuristics. There is also some nuance in how the two were compared, as Sugie et al. (2018) tested their own PSSA against their own reimplementation of the algorithm

Component	Original, King's graph	Ember, D-Wave hardware
Hardware graph	King's graph $\mathrm{KG}_{L,L}$	Chimera, Pegasus or Zephyr graph
Guiding pattern	complete graph embedding of K_{L+1} (Okuyama et al. 2016)	a complete graph embedding found by D-Wave's busclique
Initial placement	guiding pattern divided into $\lvert V(I)\rvert$ super vertices	the first $\lvert V(I)\rvert$ chains of that clique embedding
Annealing steps $t_{\max}$	7×10^7 for 102 400 hardware vertices	7×10^7 scaled in proportion to $\lvert V(H)\rvert$

Table 2 | The parts of PSSA that the Ember implementation (Macaskill-Smith et al. 2026) changed to target D-Wave hardware, adapted from the table in its repository, against the original King's graph version of Sugie et al. (2021). The swap and shift moves, the acceptance rule and the terminal search are the same in both.

behind minorminer (Cai et al. 2014), while Macaskill-Smith et al. (2026) tested D-Wave's own minorminer against their own reimplementation of PSSA, so in both papers it was the reimplemented algorithm that lost.

Thus, for the moment I use the open-access reconstructions, and of the two available I found the Ember code to be the more extensive version, and with it came an extensive and well curated library of test graphs for benchmarking, along with the scripts for five other state-of-the-art minor-embedding algorithms, minorminer, OCT-based embedding, ATOM, CHARME and D-Wave's complete graph embedding, enabling direct comparison. The one nuance was that the Ember benchmark suite was strictly embedding onto D-Wave topologies, and thus their PSSA script, which the pseudocode explicitly intends to be King's graph native, was modified to instead embed onto Chimera, Pegasus and Zephyr architectures, which are the three staple D-Wave hardware topologies and, like the King's graph, all have known embeddings of complete graphs (Sugie et al. 2021). The changes were made in the hardware graph, the guiding pattern and the initial placement, and these are listed in Table 2, adapted from the table in the Ember repository, together with how the number of annealing steps $t_{\max}$ is set.

The Ember group precisely disclosed the elements of the algorithm structure changed in the repository, so I could cleanly revert these changes and take their working code, now adapted to the King's graph lattice. I reverted the hardware graph, the guiding pattern and the initial placement to the King's graph versions of Sugie et al. (2021), and kept Ember's scaling of $t_{\max}$ with the size of the hardware. The Ember code, with this change back to the King's graph topology, is actually a complete implementation of the pseudocode with no new additions, and thus follows the exact structure and philosophy of Sugie et al. (2021) outlined in Section 2.1.

The Ember code has no way of seeing the embedding itself, so to be able to follow what the algorithm actually does, I added a visualization to the reverted code. It draws the King's graph as a faint lattice and each super vertex as a thick coloured path through it, in the style of the TRIAD figures of Choi (2011) (Fig. 1), and it can show any stage of a run, from the guiding pattern and the initial placement to the final embedding (Fig. 3). In the final embedding, the input edges are drawn as black lines between the super vertices they connect, so that one can check at a glance that every edge is embedded. The faint grey lattice underneath is only a background, showing the hardware topology itself, that is, where connections can be made and where super vertices can lie. The same style is used for every embedding figure in this report, including Fig. 2.

2.3 Improvements to PSSA

Having taken the reverted Ember code of Section 2.2 as my baseline, I made four changes to it, and together these make up the improved PSSA used throughout the rest of this work. In order of importance, these are Gibbs-weighted proposals for both the swap and the shift moves, a cheap local computation of the energy change ΔE in the Metropolis step, a geometric annealing schedule, and a C++ version of the annealing loop. It is worth noting that only the first of these really changes the quality of the search itself. The local computation of ΔE and the C++ version only reduce the wall-clock time per iteration, and the geometric schedule is a simplification which in my runs did much better than the linear schedule of the baseline and no worse than the double exponential schedule of Sugie et al. (2021). Everything else, namely the guiding pattern, the initial placement, the terminal search and the validity checks, is left exactly as it was in the baseline.

2.3.1 Gibbs-weighted swap and shift moves

In Algorithm 1 both moves are proposed blindly. Once a swap has picked its edge $(i,k) \in E(I)$, the partner j is drawn uniformly from all super vertices adjacent to $\phi(k)$ (lines 5–8), and once a shift has picked its leaf $u \in \mathrm{Leaf}[\phi(i)]$, the receiving super vertex $\phi(j)$ is drawn uniformly from all candidates possessing a leaf adjacent to u (lines 12–15). Neither choice looks at whether the move would actually help, so many of the proposals are moves which lose embedded edges and then just get rejected by the Metropolis rule. This wastes iterations, and it's worst late in the run, when the temperature is low and almost every worsening move is rejected.

The improvement is to look one step ahead before proposing. For every candidate j in the candidate set C, I compute the change in the number of embedded edges ΔE_j that the move would cause, and then draw the candidate from the Boltzmann, or Gibbs, distribution

$$P(j) = \frac{\exp\big(\Delta E_j / T(t)\big)}{\sum_{j' \in C} \exp\big(\Delta E_{j'} / T(t)\big)}, \qquad j \in C, \tag{23}$$

at the same temperature $T(t)$ as the acceptance step. Thus, moves which locally gain edges are proposed more often than moves which lose them, but every candidate still keeps a nonzero probability of being picked. The Metropolis rule at lines 17–18 is unchanged, and it still decides acceptance on the true change in E_{emb}.

Since the proposal uses the annealing temperature, it anneals together with the acceptance rule. At high $T(t)$ the weights in Eq. (23) are all close to one and the proposal reduces to the

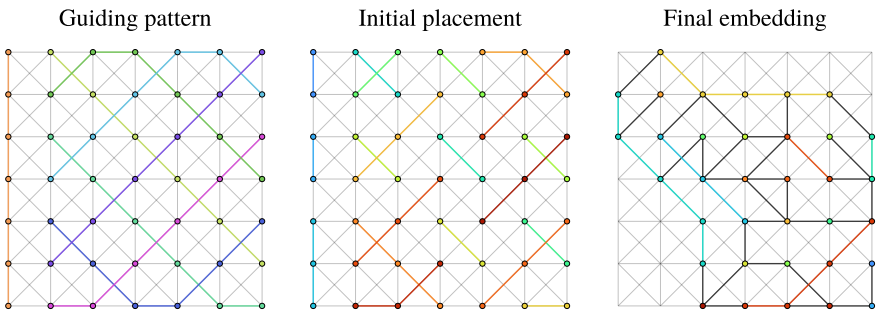


Figure 3 | The visualization added to the reverted Ember code, for an Erdős–Rényi graph with 20 vertices and 30 edges on $KG_{7,7}$: the guiding pattern (left), the initial placement (middle) and the final embedding after the terminal search (right), which uses 35 of the 49 hardware vertices. Each super vertex has its own colour, and in the final embedding the black lines are the embedded input edges.

uniform choice of the original algorithm, so the early exploration of Φ is left untouched, whereas as $T(t) \to 0$ the distribution concentrates on the best candidate and the proposal becomes greedy, which is exactly when blind proposals would waste the most iterations.

Computing each ΔE_j by recounting E_{emb} from scratch would be far too expensive, but it's also unnecessary, as a move can only change whether an input edge is embedded if that edge touches one of the two super vertices involved. So ΔE_j is found by rechecking only the input edges incident to i and j, before and after the move, which costs an amount set by the degrees of i and j and the sizes of their super vertices, and not by the size of the hardware.

To check the effect of the Gibbs proposals, I looked at two things, whether they increase the success rate on instances that are already hard for the algorithm, and secondly whether they let the algorithm actually embed larger graphs on the same hardware size. I first ran the improved PSSA with the Gibbs weighting switched on and off, keeping everything else identical, on two Erdős–Rényi random graphs and a random cubic graph, each at a hardware size L where it is hard for the algorithm to embed, with 20 seeds each and $t_{\max} = 3 \times 10^5$ (Table 3). Since the Gibbs proposals cost more per iteration, I also reran the uniform version with $t_{\max}$ scaled so that its annealing time matches that of the Gibbs version. On every graph the Gibbs version succeeds considerably more often, and giving the uniform version the same time rather than the same number of iterations makes no real difference, so the gain isn't simply down to the Gibbs version doing more work. A fourth graph, a Barabási–Albert graph with 30 nodes, was left out, as neither version could embed it.

Next, I measured the embedding threshold $\bar{V}(L)$ of Sugie et al. (2021) (Section 2.1.3) on random cubic graphs, with the Gibbs weighting switched on and off and everything else identical, over hardware sizes from $L = 6$ to 12 with 20 graphs at each size and $t_{\max} = 3 \times 10^5$ (Fig. 4). The run with Gibbs proposals wins at every hardware size, embedding about 25% more vertices on average, and giving the uniform version the matched runtime again makes no real difference.

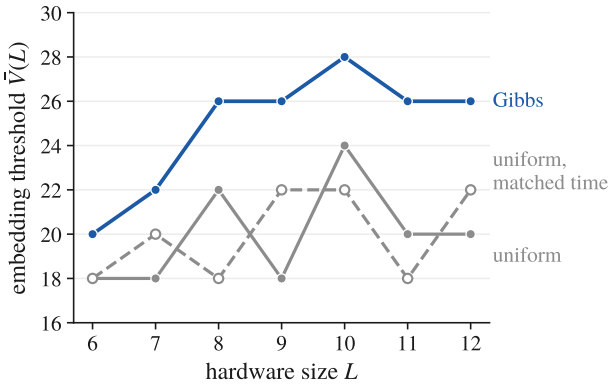


Figure 4 | The embedding threshold $\bar{V}(L)$ on random cubic graphs, the smallest number of vertices at which fewer than 19 of the 20 graphs could be embedded, against the hardware size L, with uniform and with Gibbs-weighted proposals at $t_{\max} = 3 \times 10^5$. The dashed line gives the uniform version a $t_{\max}$ scaled so that its annealing time equals that of the Gibbs version, as in Table 3. The number of vertices was increased in steps of two.

2.3.2 Computing ΔE locally

In the baseline, the ΔE at line 17 is found by recounting $E_{\text{emb}}(\phi_{\text{proposed}})$ from scratch, which means scanning every edge of H. On $KG_{L,L}$ this is roughly $4L^2$ edges per iteration, so the cost of a single iteration grows with the size of the hardware, even though each move only ever changes a handful of super vertices. The local deltas of Section 2.3.1, which the Gibbs proposals compute anyway, remove this cost entirely, as the ΔE of the chosen move is simply its local delta, and the current value of E_{emb} is kept up to date as moves are accepted. The cost of an iteration then depends only on the degrees and super vertex sizes involved in the move, and not on L. As a safeguard against the running count drifting, E_{emb} is recounted from scratch every 10^6 iterations and checked against the tracked value. Since the local delta is exact, this change leaves every acceptance decision exactly as it was, and so it only reduces the wall-clock time.

				Uniform proposals		Gibbs proposals
Graph family	$\lvert V(I)\rvert$	$\lvert E(I)\rvert$	L	$t_{\max} = 3 \times 10^5$	Matched runtime	$t_{\max} = 3 \times 10^5$
Erdős–Rényi	20	30	6	12/20	13/20	20/20
Erdős–Rényi	20	40	11	6/20	5/20	12/20
Random cubic	24	36	7	1/20	1/20	6/20

Table 3 | Successful embeddings out of 20 seeds with uniform and Gibbs-weighted proposals. Since a Gibbs step costs more than a uniform one, the matched-runtime column gives the uniform version a $t_{\max}$ scaled so that its annealing time equals that of the Gibbs version. A run counts as a success if it ends in a valid minor embedding after the terminal search.

		Time per annealing step (µs)		
L	$\lvert E(H)\rvert$	ΔE from full E_{emb}	ΔE from local edges	Speed-up
6	110	0.32	0.25	1.3×
10	342	11.5	7.75	1.5×
20	1 482	34.3	21.4	1.6×
40	6 162	47.8	6.54	7.3×
80	25 122	348	8.88	39×

Table 4 | Time per annealing step with ΔE found from a full count of E_{emb} (the baseline) or from only the edges touching the move (local ΔE), both with Gibbs proposals. The input graphs are random cubic graphs that grow with the hardware, with one vertex for about every four hardware vertices. Therefore only the hardware size changes between instances, keeping the graph structure and the typical super vertex size unchanged. The speed-up is the first time divided by the second. Each time is the best of three seeds over $t_{\max} = 5 \times 10^4$ steps.

To check this, I timed a single annealing step with the old core, which recounts E_{emb} at every iteration, and with the new core, which computes ΔE locally, on random cubic graphs whose size grows with the hardware ($|V(I)| \approx L^2/4$), so that the super vertices stay at around four hardware vertices each (Table 4). On small hardware the two are much the same, as the Gibbs proposals dominate the cost of a step there. But the recount grows with the hardware while a step with local ΔE barely does, so that by $L = 80$ the new core is 39× faster.

2.3.3 Geometric annealing

The baseline anneals with the linear two-phase schedule of Eq. (14). For simplicity I replaced this with a single geometric schedule,

$$T(t) = T_0\, \alpha^t, \tag{24}$$

with $T_0 = 6.0315$, one tenth of the value of Sugie et al. (2021), as in the reconstruction of Hung and Roychowdhury (2024), and α chosen so that the temperature falls to $T_{\text{end}} = 0.01$ at $t = t_{\max}$. The schedules $p_s(t)$ and $p_a(t)$ are unchanged. The choice of schedule doesn't matter much here, as Sugie et al. (2021) found that exponential schedules "tend to outperform" linear ones, and that single and double exponential schedules perform "identical within the bounds of statistical fluctuations". Dropping the second phase does, however, remove the reheating restart of the two-phase schedule, so in its place I run several independent annealing runs from different random seeds and keep the valid embedding which uses the fewest hardware vertices.

2.3.4 Translation to C++

Lastly, I rewrote the annealing loop of Algorithm 1, where almost all of the runtime is spent, in C++, leaving the set-up and the terminal search of Algorithm 2 in Python. The algorithm stays exactly the same, though the C++ loop changes the placement in place and undoes the change when a proposal is rejected, rather than copying the whole placement at every proposal. On a 20-node, 30-edge random graph on $\text{KG}_{6,6}$ with $t_{\max} = 3 \times 10^5$, it ran about 30 times faster than the Python loop, with both finding a valid embedding, and as this was before ΔE was computed locally, the speed-up comes from the translation to C++ alone.

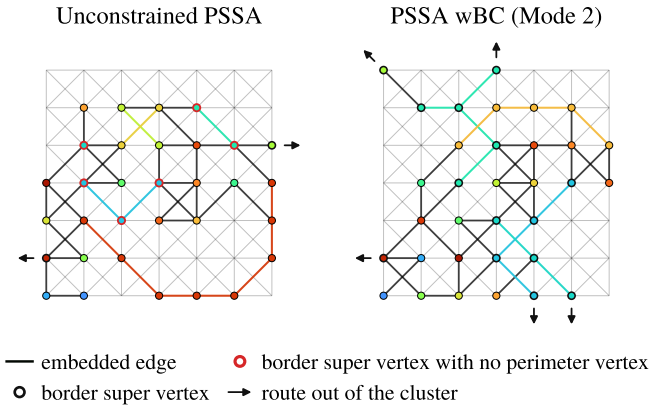


Figure 5 | The same cluster, border nodes and seed embedded with unconstrained PSSA (left) and with PSSA wBC in Mode 2 (right), for an Erdős–Rényi graph with 20 nodes and 30 edges on $\text{KG}_{7,7}$ with five border nodes chosen at random. Black lines are the embedded input edges and border super vertices are outlined, and an arrow marks the perimeter vertex through which each can be reached from outside the cluster. Without the constraint, three of the five border nodes (red) are left with no perimeter vertex and could not be connected to the rest of the network.

2.4 PSSA with border constraints

Due to my motivation of partitioning large networks into smaller manageable clusters and then embedding everything together, I could not just take each cluster and run PSSA on it to embed in whichever layout it converges to. When a network is partitioned and the cut size is nonzero, some clusters, if not all, contain nodes that are connected to nodes in other clusters. That is, they have neighbours outside of their cluster, and I call these *border nodes*. What this means is that in an unconstrained PSSA embedding, the resulting embedding could have border node super vertices away from the perimeter of the cluster, jumbled up somewhere near the centre instead. These border nodes would then not be able to satisfy their outer-cluster connections. To combat this problem I added a constraint to PSSA which ensures that border nodes, once identified by the graph partitioning (Sections 3.1 and 4.3), always lie on the border of the final cluster embedding. I call this PSSA with border constraints, or PSSA wBC.

Let $B \subseteq V(I)$ be the set of border nodes of a cluster, that is, the vertices with at least one neighbour in another cluster, which come directly out of the partitioning. Let $P \subset V(H)$ be the perimeter of the hardware, the outer ring of hardware vertices of $\text{KG}_{L,L}$, of which there are

$$|P| = 4L - 4. \tag{25}$$

The perimeter is where a cluster meets its neighbours, since when the clusters are joined (Sections 3.2 and 4.1) the cut edges between them are routed across the hardware between the cluster embeddings, and so a border node can only be connected if its super vertex can be reached from outside the cluster. The border constraint therefore asks that every border super vertex keeps at least one hardware vertex on the perimeter,

$$\text{(B)} \qquad \forall b \in B : \ \phi(b) \cap P \neq \emptyset, \tag{26}$$

alongside the three conditions (M1)–(M3) of Section 2.1.1. Since the super vertices are disjoint (M2), no two border nodes can share a perimeter vertex, and so (B) can only be satisfied when $|B| \leq 4L - 4$, which sets a lower bound on the hardware size L for a cluster with many border nodes. Unlike (M3), (B) is not treated as an objective but as a hard constraint, in the same way as (M1) and (M2), so that every placement visited during the search satisfies it. Unlike (M1) and (M2), however, (B) is not preserved by the moves automatically, so it has to be enforced with explicit checks. This takes two changes to PSSA, an initial placement which satisfies (B) from the start, and restrictions on the swap and shift moves so that no move can break it. I built the move restrictions in two versions, Mode 1 and Mode 2, which are described below.

2.4.1 Border-aware initial placement

The initial placement of Section 2.1.1 splits the guiding pattern into $|V(I)|$ pieces and hands them out in vertex order, with no regard for which vertices are border nodes, so many border super vertices start in the interior of the hardware. PSSA wBC instead hands the border nodes the pieces that already touch the perimeter first, and the remaining pieces go to the other vertices (Fig. 6). It is the same set of pieces, only handed out in a different order, so it costs nothing, and it satisfies (B) from the very start whenever there are at least as many perimeter pieces as border nodes. Only if a border node is still left in the interior is its super vertex extended out to the perimeter along the shortest path, by taking hardware vertices from the non-border super vertices in its way, without disconnecting or emptying them, which is rarely needed.

2.4.2 Mode 1: fixed border nodes

In Mode 1, one perimeter vertex of each border super vertex, taken from the initial placement, becomes its *anchor*, and it is pinned there for the entire run. The anchor can never be shifted out of its super vertex, so it is left out of the leaves offered to the shift at lines 9–16, and any swap involving a border super vertex is rejected outright, as the swap would hand the anchor to a different input vertex. The rest of a border super vertex is free to grow and shrink through shifts as normal, and the terminal search is likewise not allowed to delete an anchor.

Mode 1 satisfies (B) trivially, but in practice it proved too strict. The anchors are fixed by the initial placement rather than chosen by the search, and since every swap involving a border node is forbidden, the border super vertices can never be moved to a better position, only resized around their anchors. This matters most late in the run, where $p_s(t) \to 0$ and swaps make up almost all of the proposals, so that the more border nodes a cluster has, the more of the search is simply frozen.

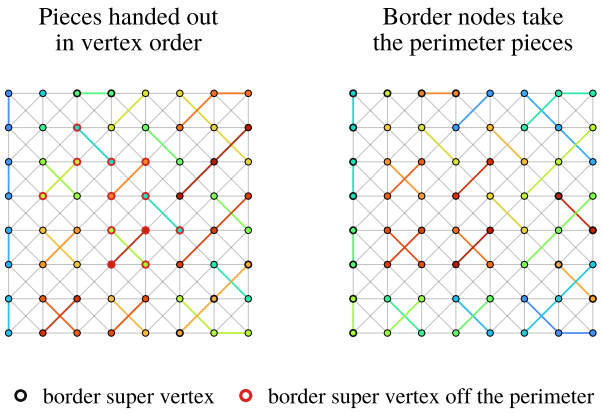


Figure 6 | The initial placement for an Erdős–Rényi graph with 30 nodes and 45 edges on $KG_{8,8}$ with eight border nodes (outlined), before any annealing. Left, the guiding-pattern pieces handed out in vertex order, which leaves six of the eight border nodes off the perimeter (red). Right, the border-aware placement of PSSA wBC, where the border nodes take the pieces that touch the perimeter, so every one of them starts on it.

2.4.3 Mode 2: free border nodes

Mode 2 keeps only what (B) actually requires, namely that each border super vertex has *some* perimeter vertex at all times, without fixing which one. For every super vertex the search keeps track of its number of perimeter vertices,

$$\tau(i) = |\phi(i) \cap P|, \tag{27}$$

which is updated whenever a move is accepted, and a proposal is rejected before the Metropolis test only if it would bring a border super vertex down to $\tau = 0$. For a shift, this means a border super vertex may only give away a perimeter vertex if it holds at least one more, so a shift of u out of $\phi(b)$ is rejected exactly when

$$b \in B, \qquad u \in P \qquad \text{and} \qquad \tau(b) = 1. \tag{28}$$

For a swap of $\phi(i)$ and $\phi(j)$, each input vertex takes over the other's super vertex, so if i is a border node the swap is only allowed when $\tau(j) \geq 1$, and likewise for j. Thus a border node can trade places with any super vertex which itself touches the perimeter, and so the border nodes are free to permute around the perimeter, swapping with each other and with the non-border super vertices that sit on it. The terminal search gets the same rule as a third condition in is_deletable,

$$\text{(p3)} \qquad u \in P \text{ and } i \in B \implies \tau(i) \geq 2, \tag{29}$$

so that the cleanup never deletes the last perimeter vertex of a border super vertex. Everything else is identical to the improved PSSA of Section 2.3, and Mode 2 is the version used for every cluster in the rest of this work.

Figure 7 follows a single Mode 2 run from the initial placement to the final embedding. Over the run the border nodes moved around the perimeter while the rest of the embedding came together, and Fig. 8 shows the same cluster embedded with Mode 1, where the pinned anchors leave ten of the thirty edges missing.

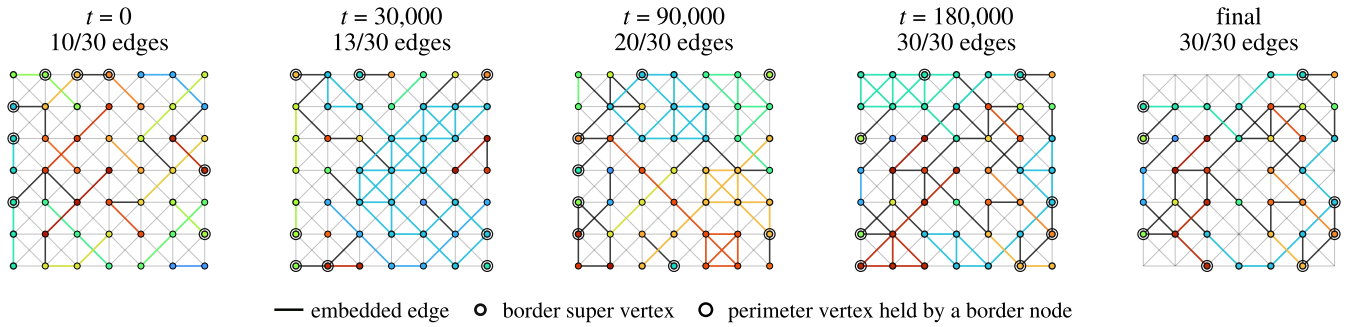


Figure 7 | Snapshots of one Mode 2 run for an Erdős–Rényi graph with 20 nodes and 30 edges on $KG_{7,7}$ with eight border nodes, at $t = 0$ (the border-aware initial placement) and at 10, 30 and 60% of t_{max}, followed by the final embedding after the terminal search. Border super vertices are outlined and the perimeter vertex each holds is ringed, and the black lines are the embedded input edges. The border nodes do not stay where they started, but move around the perimeter as the search goes on. Node numbers are left out at this size.

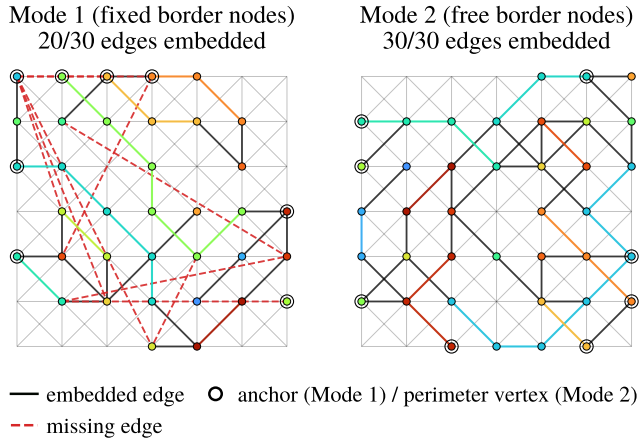


Figure 8 | Mode 1 (left) and Mode 2 (right) on the cluster of Fig. 7, with the same border nodes and seed. Ringed vertices are the fixed anchors in Mode 1 and the perimeter vertices held at the end in Mode 2, and dashed red lines are input edges that are not embedded.

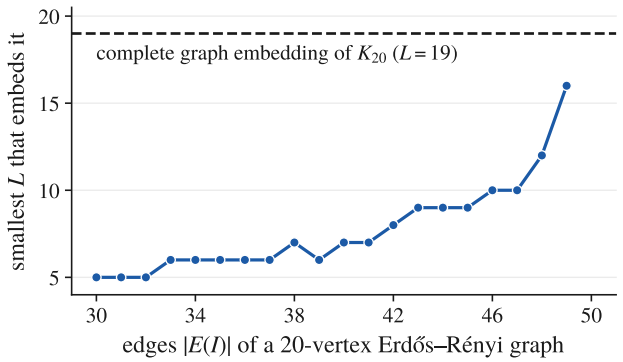


Figure 9 | The smallest hardware size L at which the improved PSSA (unconstrained) found a valid minor embedding of a 20-vertex Erdős–Rényi graph, against its number of edges, with a new random graph for each edge count. Each L was tried with 10 runs of $t_{max} = 10^6$ (3 runs from 47 edges on). At 50 edges no embedding was found up to $L = 14$, where the search was stopped, so that point is not shown. The dashed line is the complete graph embedding of K_{20}.

2.5 Results and limits

Since PSSA wBC is PSSA with extra restrictions on top, it can never do better than the improved PSSA it is built on, and so the first limit of the cluster embedding is simply the limit of PSSA itself. What PSSA struggles with most is density. Figure 9 shows this for a 20-vertex Erdős–Rényi graph as its number of edges is increased from 30 to 50. Up to about 37 edges, a mean degree of 3.7, the smallest hardware that embeds it stays at $L = 5$ or 6, but beyond that it climbs quickly, to $L = 16$ at 49 edges, which is already close to the $L = 19$ of the complete graph embedding of K_{20}, which embeds any 20-vertex graph with no search at all. This agrees with Sugie et al. (2021), who found that for Erdős–Rényi graphs with 20% edge density even the improved PSSA could not embed larger graphs than the complete graph embedding already can, so that such graphs remain embeddable only by resorting to it.

To measure how much the border constraint costs, I ran both modes on the Erdős–Rényi graph with 20 nodes and 30 edges from Table 3, choosing 0, 2, 5, 10 and 15 of its vertices at random as border nodes, over hardware sizes from $L = 5$ to $L = 9$ with 20 seeds each (Fig. 10). The result is clear-cut. Mode 1 fails in every run as soon as there are any border nodes at all, even with only two of them on hardware where the unconstrained search always succeeds, which confirms that fixing the anchors freezes too much of the search. Mode 2 on the other hand behaves as one would hope, as a few border nodes cost it very little, the success rate falls gradually as their share grows, and a slightly larger L recovers most of the loss. Even at 15 border nodes, three quarters of the vertices, Mode 2 still succeeds in about half of the runs from $L = 7$ on, and only on the smallest hardware, $L = 5$ and 6, can the fifteen border nodes not all be placed on the perimeter at the start.

I also looked at whether it matters which vertices are the border nodes, since in a real partition these are fixed by the graph rather than chosen at random. First, looking at where the super vertices of vertices of different degree end up in unconstrained runs, I found no strong preference. On average, higher-degree vertices sit slightly further from the edge of the hardware, but the effect is weak. The choice of border nodes does, however, show up in the success rate when the hardware is tight (Fig. 11). Over 20 Erdős–Rényi graphs with 20 vertices and 30 edges and five border nodes each, choosing the five highest-degree vertices as border nodes clearly hurts, while choosing the five lowest-degree vertices helps mainly on the smallest hardware. By $L = 9$ the difference disappears.

Finally, what matters for the rest of the method is how large a cluster the cluster embedding can take. To measure this directly, I embedded Erdős–Rényi graphs of 10 to 40 vertices with a mean degree of 2.5 to 6, in the same way the full pipeline does it:

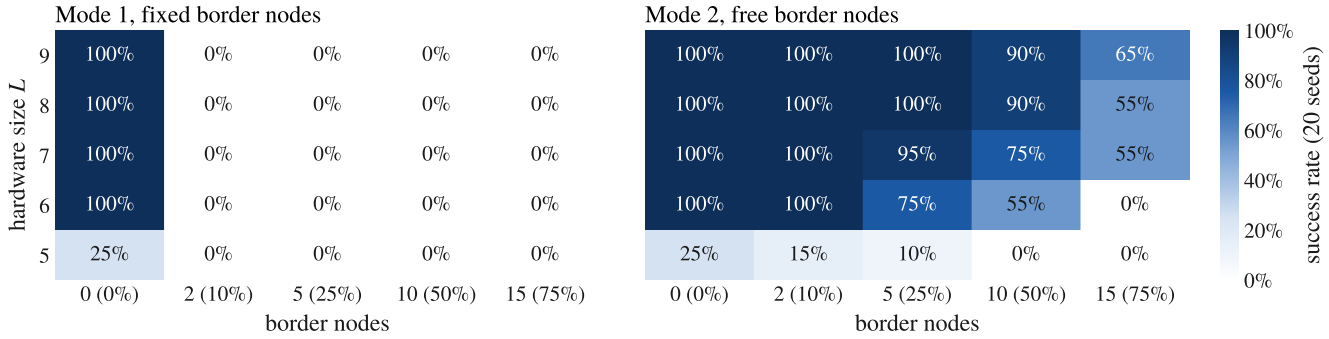


Figure 10 | Success rate of Mode 1 (left) and Mode 2 (right) against the number of border nodes and the hardware size L, on an Erdős–Rényi graph with 20 nodes and 30 edges. The border nodes are drawn at random for each of the 20 seeds, and the column with no border nodes is unconstrained PSSA. A run counts as a success only if it ends in a valid minor embedding. At 18 border nodes (90%, not shown) no run could even start, in either mode and at any L, as only 13 to 16 of the 20 pieces of the guiding pattern touch the perimeter and a corridor may not take vertices from another border node.

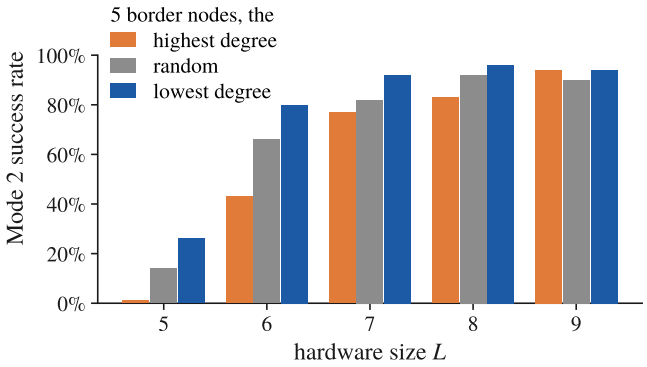


Figure 11 | Mode 2 success rate against hardware size L, with five border nodes chosen as the highest-degree, five random, or the lowest-degree vertices (ties broken by vertex number), over 20 Erdős–Rényi graphs with 20 vertices and 30 edges and 5 seeds each, so each bar is the fraction of 100 runs that succeeded. The choice of border nodes only matters when the hardware is tight.

starting from the smallest L that can hold the cluster, trying two seeds at each L, and growing L by one up to four steps above the smallest (Fig. 12). Each cluster was embedded without border constraints and with 30, 50, 70 and 90% of its vertices chosen at random as border nodes (the figure shows up to 70%), which separates the limit of PSSA itself from the cost of the constraint. Without border constraints, the limit comes from the density of the cluster, just as in Fig. 9. With 30 and 50% border nodes the limits stay about the same, and the constraint only lowers the success rate at their edge, so it costs very little. Only from 70% on do the border nodes start to set the limit themselves.

This maps out where the cluster embedding succeeds, and so what the partitioning has to produce, namely clusters sparse enough for their size to fall inside that region. On these Erdős–Rényi clusters it reaches about 15 to 40 vertices, depending on their density and on how many of their vertices are border nodes, while the clusters of a real partition are much sparser than this, as shown below. It also connects the partitioning and the cluster embedding in a second way. Figures 10 and 12 show that fewer border nodes make a cluster easier to embed, so a partition with a low border node count T (Section 4.3) is not only cheaper to route between but also easier to embed within, and the degree result adds that, when the hardware is tight, a partition whose border nodes have high degree will be harder to embed than one whose border nodes have low degree.

It is worth noting that the limit is PSSA itself, and not the perimeter or the shape of the patch. My PSSA, built on the Ember reimplementation, embeds random cubic graphs only up to about 30 to 50 vertices, where Sugie's own code reaches about a thousand, as at 50 vertices and beyond no run embedded within five minutes on any hardware size tried. The clusters of a real partition, however, are much sparser inside than the graph they come from, since the cut edges are removed, and for random cubic graphs split into clusters of up to $s = 40$ or 50 vertices their mean internal degree is only about 1.9. Thus PSSA wBC embeds clusters of $s = 50$ cleanly, and it is PSSA's own limit at around 50 vertices, not the border constraint, that caps the cluster size.

3 Divide and Conquer method

Divide and Conquer is a long-established strategy in electronic design automation, the field that builds the tools for designing electronic circuits and chips, for mapping a large network onto hardware with a fixed topology, that is, for giving each component of the network a place on the hardware and realizing each of its connections through the hardware's own available connections. When a network is far too large to be mapped as a whole, it is partitioned into clusters with as few connections between them as possible, each cluster is mapped onto its own region of the hardware chip, and the regions are then arranged, which is known as floorplanning, and the connections between them are routed. The classic example is Kernighan and Lin (1970), who split electronic circuits over several circuit boards with as few connections between the boards as possible. Breuer (1977) applied the same idea to placing the components of a chip, cutting the circuit and the chip area in two again and again with as few connections across each cut as possible, and today the same partitioning divides large computations between the processors of a parallel computer with as little communication between them as possible (Karypis and Kumar 1998). I found that the same idea carries over naturally to minor-embedding, as a graph too large for any heuristic embedding algorithm to embed at once, due to the NP-hard scaling of the configuration space, can be split into clusters small enough for the algorithm

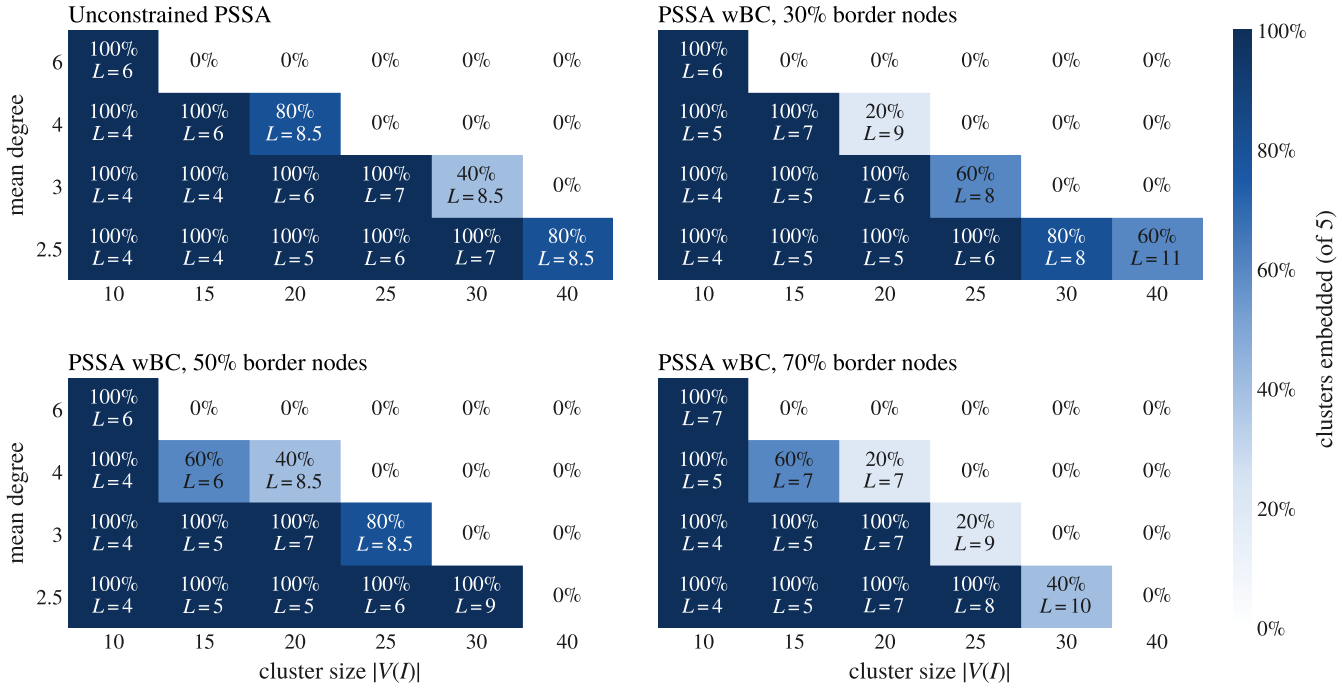


Figure 12 | How many of 5 clusters embed, by cluster size and mean degree, with plain PSSA (top left) and with PSSA wBC in Mode 2 and 30, 50 or 70% of the vertices chosen at random as border nodes. The clusters are Erdős–Rényi graphs. Each cluster was tried from the smallest hardware that fits it, L_0, up to $L_0 + 4$, with two seeds of $t_{\max} = 4 \times 10^5$ at each L, and each cell gives the median smallest L that worked. It's worthy of note that a cell at 0% indicates that no embedding was found in the tested L range, not that none exists.

to handle, each cluster embedded on its own patch of the King's graph lattice, and the patches then arranged on one shared hardware chip and joined by routing the cut edges between them.

Clearly, the objective throughout is to minimize the cut size. Every cut edge has to be routed across the hardware between two clusters according to the hardware topology, which is the part that limits the method. Furthermore, the two ends of the cut edge become border nodes, which PSSA wBC must hold on the perimeter of their patches, so a smaller cut also makes the clusters easier to embed (Section 2.5) as a smaller percentage of the logical nodes are border nodes. Every cluster has to stay within what PSSA can embed, which is limited by its number of vertices, its density and its fraction of nodes that are border nodes (Figs. 9 and 12). Thus, the number of vertices per cluster acts as a constraint on the min-cut partitioner by setting a maximum cluster size. As for the density, this is graph specific and hasn't yet been imposed as a parameter or constraint in the partitioner. The method thus runs in three steps, which Fig. 13 follows on the 100-vertex random cubic graph example, first the min-cut partitioning, then the embedding of every cluster with PSSA wBC (Section 2.4) so that its border nodes lie on the perimeter of its patch ready to be connected, and finally the joining of the patches into a full minor-embedding of the network.

3.1 Min-cut partitioning

Rather than rely on a single partitioner, my approach runs several and keeps the best. It first tries the smallest number of clusters k that keeps every cluster within the maximum cluster size, and the next few above it, since a slightly larger number of clusters can sometimes follow the nature of the graph better and result in a smaller cut size. For each number of clusters it runs two partitioning methods, spectral clustering (Ng et al. 2002) and recursive Kernighan–Lin bisection (Kernighan and Lin 1970), and since both start from a random initial guess, it runs each of them several times from different random starts, ie. different seeds. Every candidate is then refined with Fiduccia–Mattheyses (Fiduccia and Mattheyses 1982), which moves single vertices to a neighbouring cluster whenever the move reduces the cut size, in repeated rounds over all vertices until no single move helps anymore, all while keeping every cluster within its size limits. The partition with the lowest cut is passed on to the cluster embedding. On the example graph, spectral clustering split the 100 vertices into 4 clusters with only 24 cut edges among them (Fig. 13a).

3.2 Placing and routing the clusters

Once every cluster is embedded on its own patch (Fig. 13b), the patches have to be placed together on one hardware chip and every cut edge routed between them. These are exactly the floorplanning and routing steps of VLSI physical design, the stage of chip design that lays a circuit out on the chip, so I took a standard tool from that field for each. A B*-tree floorplanner (Chang et al. 2000) places the patches, searching by simulated annealing for a compact layout in which the two ends of every cut edge lie close together, turning and mirroring the patches where it helps. PathFinder (McMurchie and Ebeling 1995), the classic router of programmable chips (FPGAs), then routes every cut edge along a path of free hardware vertices. At first the routes may share vertices, but in every round the vertices that several routes compete for become more expensive, so the routes negotiate among themselves, and given enough room every cut edge ends up with a path of its own. It is worth noting that these were simply the first tools I chose, as a quick way to

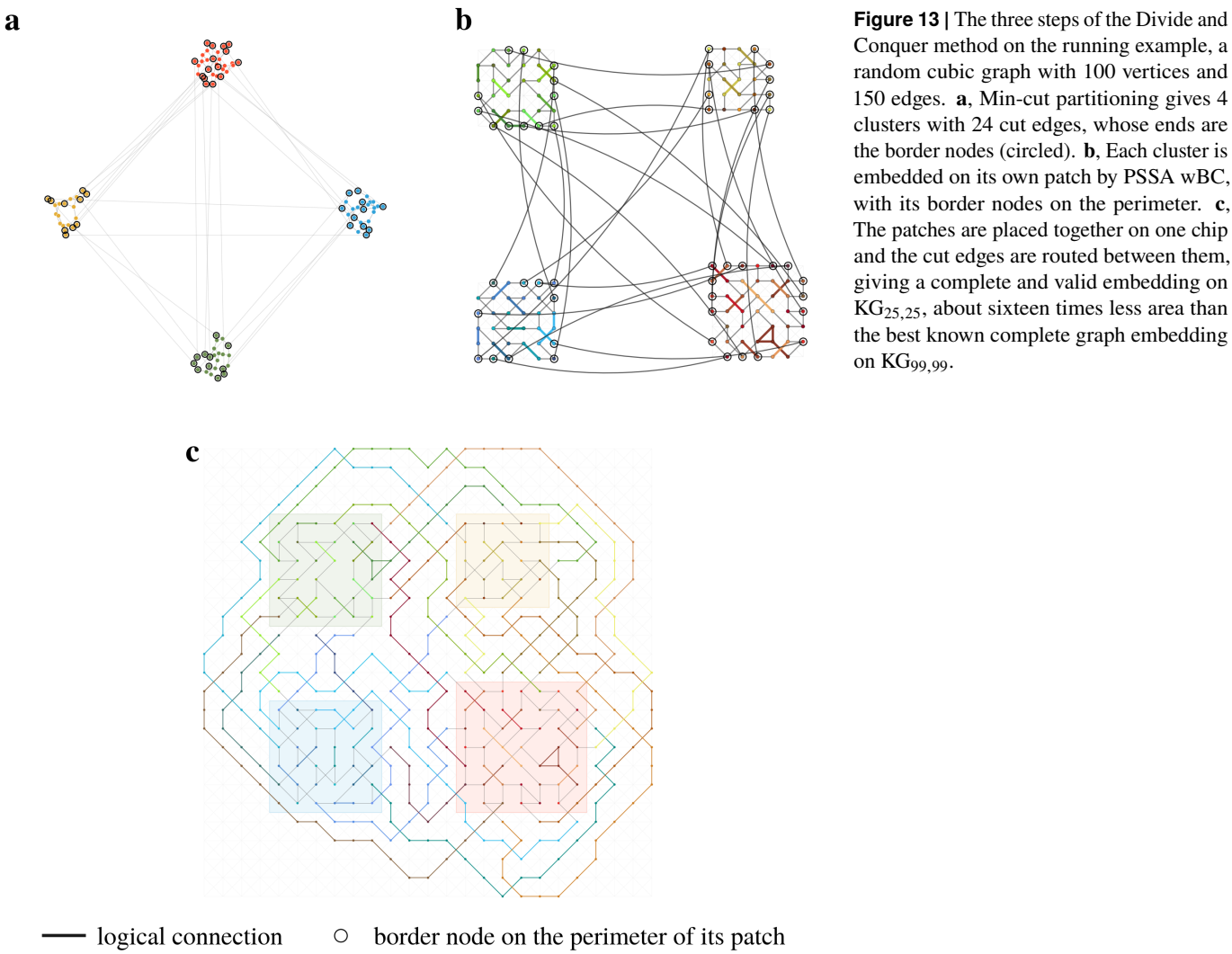


Figure 13 | The three steps of the Divide and Conquer method on the running example, a random cubic graph with 100 vertices and 150 edges. **a**, Min-cut partitioning gives 4 clusters with 24 cut edges, whose ends are the border nodes (circled). **b**, Each cluster is embedded on its own patch by PSSA wBC, with its border nodes on the perimeter. **c**, The patches are placed together on one chip and the cut edges are routed between them, giving a complete and valid embedding on $KG_{25,25}$, about sixteen times less area than the best known complete graph embedding on $KG_{99,99}$.

show that the clusters can be joined at all, and little work went into developing them.

This works because joining the clusters is a much easier problem than embedding the graph. The hard part, deciding where every vertex goes and how its super vertex is shaped within a tight space, has already been done by PSSA on the small clusters, and what is left is to route one path per cut edge between patches that are already fixed. This is easy as long as there is enough free hardware between the patches, so the joining starts with narrow corridors between them and widens the corridors until every cut edge routes. The method thus trades some compactness for more reliability.

3.3 Results and limits

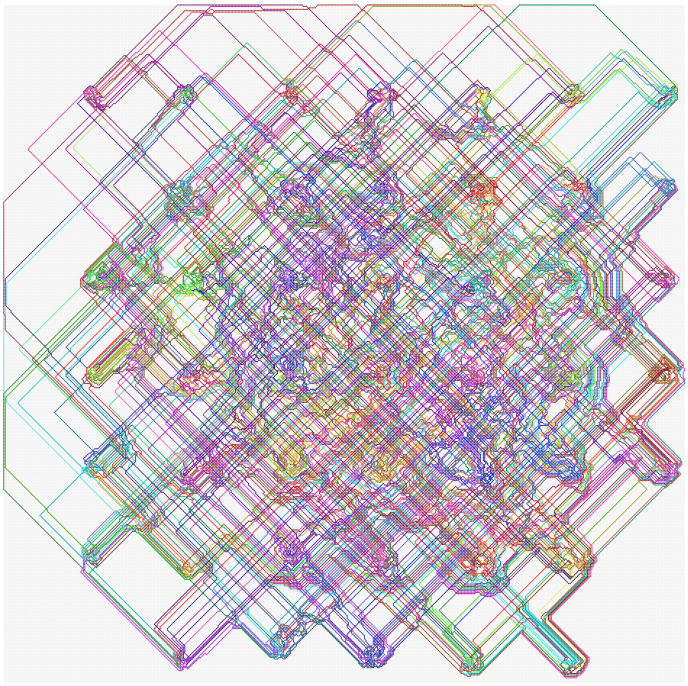
The method extends a standalone heuristic embedding algorithm, as it embeds several clusters where a standalone algorithm embeds one, and so it carries PSSA well beyond the 30 to 50 vertices my implementation manages on its own (Section 2.5). The example graph, a random cubic graph with 100 vertices and 150 edges, embeds completely and validly on $KG_{25,25}$ at about sixteen times less area than the complete graph embedding on $KG_{99,99}$ (Fig. 13c).

To see how far the method reaches, I ran it on random cubic graphs from 100 to 1,500 vertices, ten graphs at each size up to 1,000 and one at each size above, rerunning the joining with a new random seed until it succeeded, for up to 20 tries per graph. I found that every graph up to 700 vertices embedded, nearly always at the first try. From 800 vertices on the method became uncertain, as the success rate of a single try fell to about a half, so many graphs needed several tries and three of the thirty between 800 and 1,000 were never embedded. Beyond 1,100 vertices successes became rare, and on the 1,500-vertex graph none of its tries succeeded, though each failed by only a few cut edges. The limit is thus a soft one, where the chance that a single try succeeds falls as the graph grows, rather than a size at which the method stops.

The cut size is what becomes the limitation of the method as n grows. Every cut edge needs its own route through the corridors between the patches, so the lattice grows with the number of cut edges, and random cubic graphs are the notorious case for this as they are good expanders, ie. well connected throughout, so there is inherently no community structure for the graph partitioner to exploit and the cut size is large. Their cut thus grows linearly with n, and with a large coefficient, as in my runs about a third of all edges were cut at 1,000 vertices, 485 of its 1,500 edges. In practice, L follows the cut closely, as I measured $L = 471$ for those 485 cut edges, so L grows linearly with n and the area grows quadratically, and with every cut edge added the joining becomes harder.

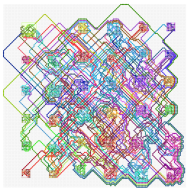
A graph with planted community structure behaves very differently. A bipartite stochastic block model, ie. a random bipartite graph made of communities that are densely connected inside and only sparsely connected to each other, keeps its cut

a



$KG_{471,471}$

b



$KG_{127,127}$

Figure 14 | The cost of Divide and Conquer is its cut, on two graphs of roughly the same size drawn to the same scale. **a**, A random cubic graph with 1,000 vertices and 485 cut edges, on $KG_{471,471}$, about 4.5 times less area than the complete graph embedding on $KG_{999,999}$, near the best of the ten graphs of this size that I ran, whose median was 3.4 times (Fig. 22). **b**, A bipartite stochastic block model with 1,014 vertices and only 159 cut edges, on $KG_{127,127}$, about 64 times less area than the complete graph embedding on $KG_{1013,1013}$.

small. In my runs, one with 1,014 vertices had only 159 cut edges and embedded on $KG_{127,127}$, about 64 times less area than the complete graph embedding, against $KG_{471,471}$ for a random cubic graph of almost the same size (Fig. 14).

In fact, on expanders like random cubic graphs, no method can do better than L growing linearly with n, which follows from the treewidth $\mathrm{tw}(G)$ of a graph G, a standard measure of how far it is from being a tree. A minor never has a larger treewidth than its host, and the King's graph $KG_{L,L}$ has treewidth at most $2L-1$ (Sugie et al. 2021), so a graph G can only be embedded in $KG_{L,L}$ if

$$\mathrm{tw}(G) \;\leq\; 2L-1.$$

Random regular graphs, and so random cubic graphs, are almost always expanders, and the treewidth of an expander grows linearly with its number of vertices (Grohe and Marx 2009). Thus for every minor-embedding of a random cubic graph, whether by the complete graph embedding, by PSSA, by Divide and Conquer or by the fallback method, L grows linearly with n and the area grows quadratically, and the methods can only differ by a constant factor. The law fixes how L scales, but not its constant. For a random cubic graph, let $L_{\min}$ be the smallest L on which it embeds. The law bounds it from below, and the complete graph embedding, which always succeeds on $KG_{n-1,n-1}$, from above, so the optimal side and area are bounded by

$$\begin{aligned} \Omega(n) \;&\leq\; L_{\min} \;\leq\; n-1, \\ \therefore\quad \Omega(n^2) \;&\leq\; L_{\min}^2 \;\leq\; (n-1)^2, \end{aligned}$$

and the optimal area grows as $\Theta(n^2)$. Its constant is, as far as I could find, still unknown.

Since no method escapes this on expanders, the methods compete on the constant factor, which determines how compact the embedding is, and consequently how close to the optimal embedding we can get. What limits Divide and Conquer is not its compactness, but its tractability as n scales to larger networks. Critically, the joining is itself a large-scale minor-embedding problem, in which the patches play the role of the vertices and the cut edges the role of the edges. Thus, whether it is done with these tools, with an adapted PSSA or with any other heuristic embedding algorithm, it remains a search on an NP-hard problem whose configuration space grows exponentially with the cut size, and no algorithm is known that solves it in polynomial time. Regardless of how much better it is than standalone PSSA or any other method, such a search can never be guaranteed to succeed in an exponential search space, and the runtime quickly becomes intractable as the graph grows. In my runs, a single attempt at joining took about a second at 100 vertices but about five minutes at 1,400, while the chance that the attempt succeeded fell towards zero. Divide and Conquer thus carries PSSA much further, but large-scale minor-embedding by search remains intractable. With the goal of scalable hardware implementation, I continued to look further, and having now identified that the main limitation is the cut edge routing, this time I looked for a way of routing the cut edges that needs no search at all, works for an arbitrary cut, and still gives a more compact embedding than the complete graph embedding. This is the fallback method of Section 4.

4 Fallback method

As of now in the field, when a heuristic minor-embedding algorithm fails on a graph, the standard fallback is the complete graph embedding, which embeds any graph with n vertices on $KG_{n-1,n-1}$ with no search at all, as it's a closed-form complete graph pattern. Every pair of its super vertices is adjacent (Section 2.1.1) at the price of a hardware size scaling as $(n-1)^2$. Since the size scaling is only dependent on the number of nodes n, the embedding is blind to network sparsity or any of its graph properties and is thus a very harsh and wasteful embedding method for sparse network graphs. Divide and Conquer is exactly such a heuristic, as its last phase of routing clusters

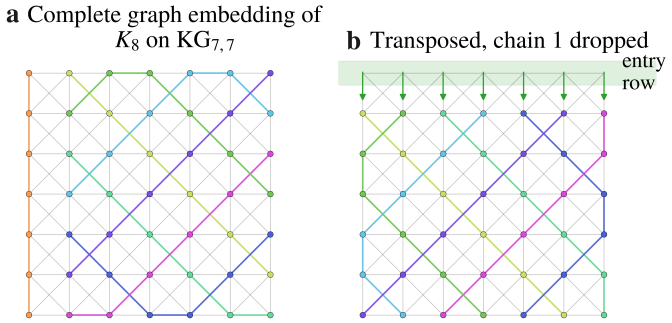


Figure 15 | The routing structure of the fallback method, drawn with my visualization. **a**, The complete graph embedding of K_8 on $KG_{7,7}$ of Okuyama et al. (2016), the guiding pattern of PSSA. **b**, The same pattern with its first super vertex dropped and transposed, so that all the remaining logical nodes lie along one row, below a free entry row (green) through which the lanes from the clusters enter from above. All-to-all connectivity is preserved, so any cut edge between two border nodes is realized where their chains meet.

together is a method that doesn't scale to large problem sizes, and what large graphs need is a way of routing the cut edges between the clusters that is guaranteed for a cut of any size and any distribution (Section 3.3). The idea of the fallback method is to solve this routing problem just as one would handle any problem a heuristic fails at, with the complete graph embedding, but only over the T border nodes since only they have to reach other clusters while every other vertex stays inside its own. The complete graph embedding of the border nodes thus becomes a routing structure between the clusters, in which every cut edge is realized without having to be searched for, while the rest of the graph is embedded compactly inside the clusters by PSSA wBC. It is in this way that the method relies on the best known complete graph embedding in order to beat it.

The method runs in the same three steps as Divide and Conquer, but the last phase of routing clusters together is now a closed-form construction with no search at all, which always succeeds (Section 4.1). This routing structure acts like a plaza where the border node ferromagnetic chains meet to realize their logical connections. In practice, the dimensions of the embedding are set almost entirely by the number of border nodes T, through its routing structure (Section 4.2), so the partitioning minimizes T instead of the cut (Section 4.3), which is where I found that minimizing the cut in the graph partitioning stage is in fact the wrong goal for this method. Together, the three steps embed a random cubic graph with a million vertices on 2.76 times less area than the current best known complete graph embedding, and an Erdős–Rényi graph of the same size on 4.43 times less, in linear time (Section 4.4).

4.1 The construction

Every cut edge is realized inside the routing structure so the border nodes of every cluster first have to reach it reliably, from wherever their cluster lies. I went with the most straightforward approach and chose to bring all of them in from one side, and the construction I finally chose is a 2D grid of the patches of the clusters, placed above the routing structure, about $\sqrt{k}$ patches wide and $\sqrt{k}$ deep, where $k = n/s$ is the number of clusters. The patches are placed in the grid in order of size, from the smallest in the top row to the largest in the bottom row, so that every row holds patches of a similar height. Every border node leaves its embedded cluster through the perimeter vertex that PSSA wBC keeps for it (Section 2.4) and runs along its own lane, through the corridors of space between the columns of clusters and the bands between their rows, down to the routing structure below. Before this I tried two simpler versions. The first lined all the patches up in a single row, which always came out wider than the routing structure below it, as its width is T plus the widths of all k patches, which was approximately $n/\sqrt{s}$; an excess that grows linearly with n and never fades. In contrast, the excess width of the grid layout grows only like $\sqrt{n/s}$ and becomes negligible for large n (Section 4.2). The second idea used the TRIAD of Choi (2011) (Fig. 1) as the routing structure, which was the natural choice as it has an entry for every border node along one side by design, but at $2T - 2$ wide for T border nodes it is about twice as wide as the routing structure I finally chose.

For the routing structure I chose the square pattern of Okuyama et al. (2016), the current best known complete graph embedding on the King's graph, which embeds K_{L+1} on $KG_{L,L}$ and is the guiding pattern of PSSA (Section 2.1.1), with two changes (Fig. 15). In that pattern the first super vertex fills the whole first column, and every other super vertex is a diagonal chain which starts next to it in the second column and crosses the lattice. Dropping the first super vertex removes that first column and opens up an entrance along which every other super vertex can be reached. Then, transposing the pattern turns this column into a row so that every chain can be entered from above, purely for the goal of a clean layout. This fit perfectly with my goal of having all the connections on one side, as every border node can enter its own chain from the grid of patches above, and for T border nodes the routing structure is thus T chains in a region T wide and $T - 1$ deep, thus there is zero excess width on the routing structure. The routing structure always works because of its all-to-all connectivity, as dropping the first super vertex leaves all the other nodes fully connected, and the transpose also preserves the all-to-all structure, so the modified pattern is itself a complete graph embedding of the T border nodes. The routing works for a cut of any size and any distribution, including many cut edges between the same two clusters, which makes it a universal solution to the routing problem I ran into in Divide and Conquer (Section 3.3).

With the routing structure in place, what remains is to bring every border node to the entrance of the structure, along a lane of hardware nodes down through the open corridors and bands, each sized to exactly the number of lanes that pass through it, and then through a funnel below the grid to enter the structure, with the lanes ordered so that no two ever meet (Fig. 16c). The last phase of routing clusters together is thus a closed-form construction with no NP-hard search, as every position and every lane follows directly from the partition and the sizes of the patches. Figure 16 follows it on the running example of Fig. 13, the random cubic graph with 100 vertices, which at $s = 30$ embeds on $KG_{63,63}$, about 2.5 times less area than the complete graph embedding on $KG_{99,99}$, while Divide and Conquer fits this small graph on $KG_{25,25}$, which Section 5 returns to. Every step of the method is thus guaranteed, as the routing always succeeds. Also, in practice, every cluster is embedded with its border nodes on the perimeter of its patch, by PSSA wBC (Section 2.4) or, should it ever fail, by the complete graph embedding of K_s on $KG_{s-1,s-1}$, which embeds any cluster and in which every super vertex reaches the edge of the lattice. Such a patch is larger, but in practice even with significant failure, the resulting

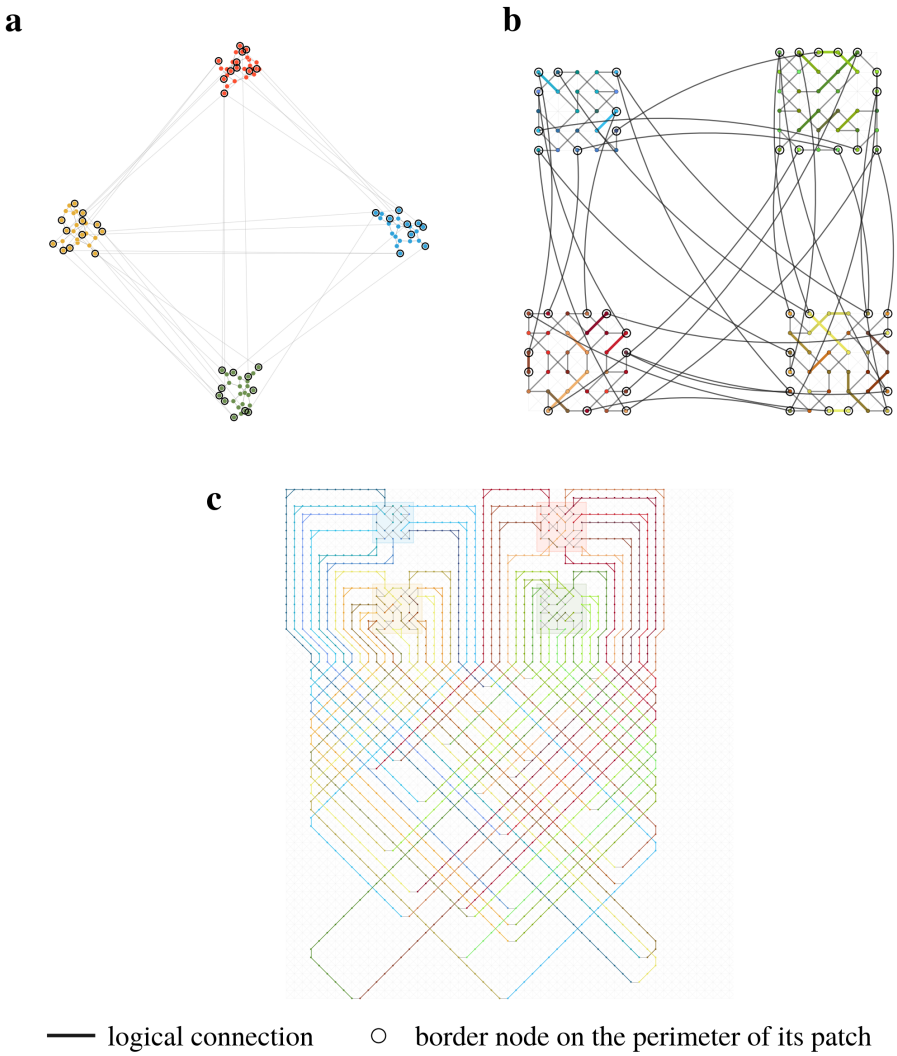


Figure 16 | The fallback method on the running example, the random cubic graph with 100 vertices of Fig. 13, at $s = 30$. **a**, Partitioning for the fewest border nodes gives 4 clusters with 43 border nodes (circled) and 28 cut edges. **b**, Each cluster is embedded on its own patch by PSSA wBC, with its border nodes on the perimeter. **c**, Every border node runs down its own lane to its own chain of the routing structure, where every cut edge is realized, giving a complete and valid embedding of the whole graph on $KG_{63,63}$, about 2.5 times less area than the complete graph embedding on $KG_{99,99}$. Figure 23 plays the same steps live, on a random cubic graph with 960 vertices.

excess grows more slowly than T, so they do not change how the embedding scales.

4.2 The size of the embedding

Where the size of the complete graph embedding is set by the number of vertices alone, the size of the fallback embedding is set mostly by the connections between the clusters, and it can be read off the construction exactly. From top to bottom, the lattice holds the grid of patches, about $\sqrt{n/s}$ rows of them, then the funnel, and then the routing structure, T wide and about T deep. The height, T plus the rows of the grid and the funnel, is thus always the larger side. To turn this into a formula, I make two approximations, that every cluster has the same size s and the same share of border nodes, T/n, and that each of the $\sqrt{n/s}$ rows of the grid adds about the same height c to the side, its tallest patch, its bands and its share of the funnel. These approximations tend to hold well for large n. The side is then

$$L \;=\; T + c\sqrt{n/s}\,,$$

where T is the routing structure and $c\sqrt{n/s}$ the grid and the funnel above it, and the area against the complete graph embedding on $KG_{n-1,n-1}$ is

$$\frac{L^2}{(n-1)^2} \;\approx\; \left(\frac{T}{n}\right)^2 \left(1 + \frac{c}{(T/n)\sqrt{ns}}\right)^2 .$$

The row height c is measured from the runs rather than derived from first principles, and it's about 38 at $s = 50$ on random cubic graphs, and with it the formula agrees closely with the measured embeddings for large graphs (Fig. 17). I found that with c measured on a random cubic graph with 100,000 vertices, the formula predicted the side of one with a million vertices to within 0.002%. For smaller graphs the approximations hold less well, as each row of the grid holds only a few patches, so its height is smaller and varies more, while for large graphs it settles to a steady c. Clearly, the T term dominates, as everything else grows only like $\sqrt{n/s}$, so the side tends to T and the area to $(T/n)^2$ of the complete graph embedding. Thus for large graphs the size of the embedding is set by T alone, and so I realized the goal of the partitioning stage should be to minimize T.

4.3 Graph partitioning to minimize border node count

Since T sets the size of the embedding, the goal of the graph partitioning stage is clearly to minimize T, an objective which, as far as I could find, has not been tackled anywhere in the partitioning literature where graph partitioners usually minimize the cut or the communication volume of a graph. Thus I had to approach it myself, and in doing so I found empirically that minimizing the cut is the wrong goal for this method, as the two objectives are in direct opposition. The link between them is that every cut edge has two ends, each on a border node, so for a given cut, T is smallest when the cut edges are gathered onto

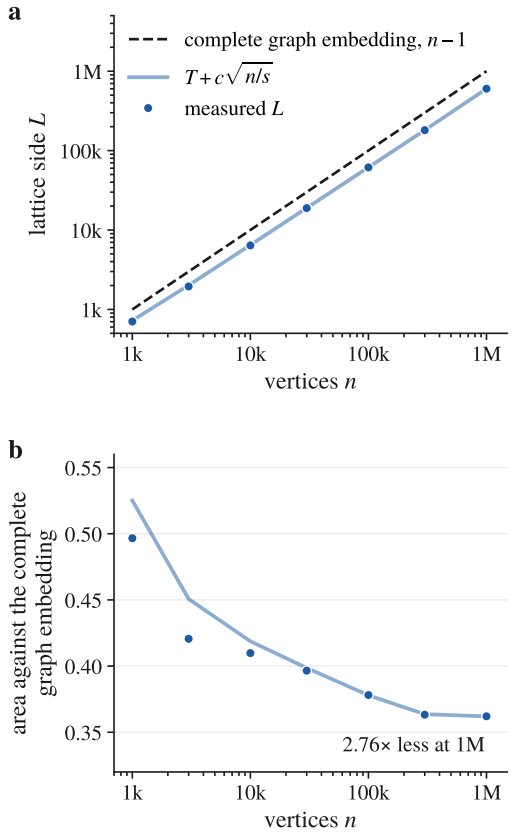


Figure 17 | The size of the fallback embedding of random cubic graphs at $s = 50$, measured on one graph at each size up to a million vertices (points) and from $L = T + c\sqrt{n/s}$ with $c = 38$ (lines). **a**, The side L. **b**, The area against the complete graph embedding, 2.64 times less at 100,000 vertices, 2.75 times less at 300,000 vertices, and 2.76 times less at a million vertices.

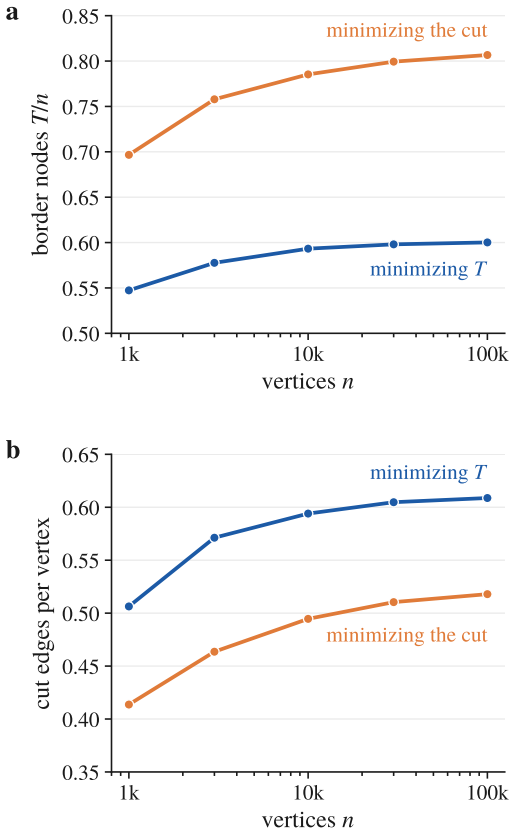


Figure 18 | Minimizing the cut and minimizing T are opposing processes. Random cubic graphs partitioned by Mt-KaHyPar at $s = 50$ with only its objective switched. The data points are taken as the mean of three graphs per point. **a**, The share of vertices that are border nodes, which keeps rising when the cut is minimized and flattens at about 0.6 when T is. **b**, The cut edges per vertex, which are fewer when the cut is minimized.

as few border nodes as possible, which a cut partitioner has no reason to do.

Figure 18 shows this on random cubic graphs, partitioned at $s = 50$ by the same partitioner, Mt-KaHyPar (Section 4.3.1), with only its objective switched between the cut and T. Minimizing the cut does give fewer cut edges, but it puts ever more vertices on a border, from 70% of them at 1,000 vertices to 81% at 100,000, while minimizing T flattens at 60%, which makes the routing structure 1.6 to 1.8 times smaller in area.

Why T/n flattens, at what value, and how it scales with n and with s, I could find almost nothing about anywhere, so these results are all empirical and I leave the question open (Section 6). To minimize T, I built two partitioners for this new objective, Mt-KaHyPar on a hypergraph whose cut-net objective equals T exactly (Section 4.3.1), and GGP + T-FM, whose refinement minimizes T directly (Section 4.3.2). It is worth noting that, as I had little prior knowledge of graph partitioning and was working under a time constraint, both partitioners were suggested to me by AI (Claude), which pointed me towards hypergraph partitioning and helped me put GGP + T-FM together. This part of the work is also the most recent, and I have not yet had the time to develop it in depth, as the two partitioners were built mainly as a proof of concept for the fallback method. As of now, partitioning for minimizing border node count remains very open for future work (Section 6).

4.3.1 Mt-KaHyPar

Mt-KaHyPar, the Multi-Threaded Karlsruhe Hypergraph Partitioner, is an established, state-of-the-art multilevel partitioner which has been run on hypergraphs with up to 140 million vertices (Gottesbüren et al. 2022). Multilevel means that it first coarsens the graph by repeatedly merging nearby vertices, then partitions the small coarsest version directly, and finally undoes the merges step by step, refining the partition at every level. It partitions hypergraphs, in which an edge, called a net, can join any number of vertices rather than just two. In the same way a wire in a chip can join one output to several inputs, which makes hypergraph partitioning the standard tool in chip design and in parallel sparse matrix computation. Of the objectives it minimizes natively there is the cut-net objective which counts the nets that span more than one cluster, the direct analogue of the cut. There is also the km1 objective which counts, for every net, the clusters it touches minus one, which measures communication volume in parallel computing and differs from T, as a border node touching three clusters counts once in T but twice in km1.

To make Mt-KaHyPar minimize T, I build from the graph G a hypergraph H with one net per vertex v, made of v and all its neighbours, ie. its closed neighbourhood,

$$N[v] = \{v\} \cup N(v). \tag{30}$$

As v always lies in its own cluster, its net spans more than one cluster if and only if some neighbour of v lies in another one, which is exactly when v is a border node, and so

$$\begin{aligned} \text{cut-net}(H) &= \#\{\text{cut nets}\} \\ &= \#\{\text{border nodes of } G\} = T. \end{aligned} \tag{31}$$

Minimizing the cut-net objective on H is therefore exactly minimizing T on G, with the partitioner itself left unchanged. Building one net per vertex from its closed neighbourhood is a known construction, called the row-net model of Çatalyürek and Aykanat (1999) from the field of parallel sparse matrix computation, but there it is used with the km1 objective, and I have not found it used anywhere to make the cut-net count equal T.

4.3.2 GGP + T-FM

GGP + T-FM actually takes the opposite approach by changing the objective of the partitioner rather than its input. It's a simple twist on the most classic method of graph partitioning, going back to Kernighan and Lin (1970), which first builds a partition and then refines it. The scheme is still at the heart of today's state-of-the-art partitioners, as METIS builds its first partition by growing clusters in the same way, refines it with the same kind of moves, and then refines again at every level as it undoes its coarsening (Karypis and Kumar 1998). GGP + T-FM is flat, ie. it runs the scheme once on the full graph, with no coarsening. First, graph growing partitioning (GGP) grows the clusters one at a time from a starting vertex by a breadth-first search, up to a target size a little below the maximum cluster size s, so that the refinement has room to move vertices later. Then a Fiduccia–Mattheyses refinement (Fiduccia and Mattheyses 1982) moves single vertices between clusters whenever the move lowers the objective. Natively this objective is the cut, and the one twist is that I rewrote it to be T, hence the T-FM label.

The rewritten objective rests on one counter per vertex, $\text{out}(v)$, the number of neighbours of v outside its own cluster. A vertex is a border node exactly when its counter is positive, so

$$T = \#\{\, v : \text{out}(v) > 0 \,\}. \tag{32}$$

When a vertex v moves into another cluster, only its own counter and those of its neighbours change, so every step is local, and the change in T of a move is read off v and its neighbours alone, without ever recounting T. Between moves that change T equally the one that lowers the cut more is chosen. As of now, the refinement only accepts moves that lower the objective so it stops at the first partition that no single move improves, and letting it accept worse moves for a while, as canonical FM does, is a possible way to improve it (Section 6). Regardless, the locality is what makes GGP + T-FM so light and fast. It is a purely local algorithm, so each pass over the graph takes linear time. Mt-KaHyPar refines locally too, but it first builds a hierarchy of ever coarser graphs, which gives it a global view of the graph and better partitions, at a cost in time and memory. For this partitioner, the network graph itself is stored in compressed sparse row form as just two integer arrays, one listing the neighbours of every vertex one after another and one marking where each vertex's list begins. At $s = 50$, GGP + T-FM partitions a random cubic graph with a million vertices in about 25 s on a single core on my laptop, and one with ten million in under six minutes, both with the same resulting T/n of 0.63.

4.3.3 Comparison of the two partitioners

I compared the two partitioners on random cubic graphs of up to 100,000 vertices and over a range of cluster sizes, with both called exactly as the fallback calls them. Mt-KaHyPar gives fewer border nodes throughout, so I used it for the fallback runs (Section 4.4), with the much faster GGP + T-FM perhaps left as the lighter alternative for very large graphs in the future.

4.4 Results

The fallback method offers reliable, guaranteed minor-embedding in linear time that is more compact for sparse network graphs than the current best known complete graph embedding, and so wastes less hardware space than the current best approach in the field. I have run it on graphs of up to a million vertices on my laptop with 8 cores, and in principle it reaches far larger. Every embedding in this section is a complete and valid minor-embedding, which I checked directly against the conditions (M1)–(M3) of Section 2.1.1, so that every chain is connected, no two chains share a hardware vertex and every edge of the graph is realized. Just as for the successes of a heuristic (Section 1.4), such a check leaves no doubt, and here it confirms the guarantee of the construction at every size I ran, up to a million vertices. In every case the checked embedding had exactly the side L that the construction gives. In practice, almost all of its runtime goes into embedding the clusters with PSSA wBC, and as the clusters are independent of each other, larger graphs only really need more cores running in parallel. On random cubic graphs, the staple benchmark graphs of p-computing and quantum annealing and the hard case of Section 3.3, a graph with 100,000 vertices at $s = 50$ embeds on $\text{KG}_{61489,61489}$, with 2.64 times less area than the complete graph embedding on $\text{KG}_{99999,99999}$, one with 300,000 vertices embeds on $\text{KG}_{180817,180817}$, with 2.75 times less area, and in general the factor grows with the size of the network (Figs. 17 and 20). As the overhead of the grid fades, this tends to the floor $(T/n)^2 \approx 0.357$, about 2.8 times less area. The area keeps falling with n even though T/n has plateaued (Fig. 19), as the share of the grid of clusters, $c\sqrt{n/s}$ against T, fades only slowly, like $1/\sqrt{n}$. At a million vertices, the scale of the largest probabilistic computers today (Aadit et al. 2026), a random cubic graph embeds on $\text{KG}_{601714,601714}$, with 2.76 times less area than the complete graph embedding (Fig. 17).

The method does even better on sparse Erdős–Rényi graphs, which also make up one of the standard benchmark families of minor-embedding (Macaskill-Smith et al. 2026). An Erdős–Rényi graph with 10,000 vertices, generated with mean degree 3, embeds at $s = 50$ on $\text{KG}_{5145,5145}$, with 3.8 times less area than the best known complete graph embedding, against 2.44 times for a random cubic graph of the same size (Fig. 20). In my runs, one with 30,000 vertices reached 3.9 times less area, on $\text{KG}_{15237,15237}$, one with 100,000 vertices 4.12 times, on $\text{KG}_{49286,49286}$, and one with a million vertices 4.43 times, on $\text{KG}_{475158,475158}$, so here too the gain grows with size, as the formula of Section 4.2 predicts, since with growing n the routing structure dominates the contribution of the grid of clusters.

I found that the cluster size s has a significant impact on final embedding size. A larger s lowers T/n, which on a random cubic graph with 100,000 vertices keeps falling slowly but steadily with s, from 0.63 at $s = 30$ and 0.60 at $s = 50$ to 0.54 at $s = 1{,}000$. A larger s also makes the rows of the grid of clusters taller, though added height to the grid of clusters becomes

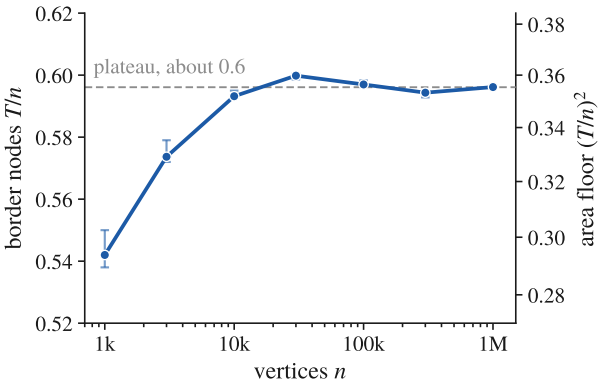


Figure 19 | The share of border nodes, T/n, of random cubic graphs partitioned by Mt-KaHyPar at $s = 50$, the median of five graphs at each size with bars for their range. It rises up to about 10,000 vertices and plateaus at about 0.6 from there to a million. The right axis gives the floor $(T/n)^2$ that the area tends to as n grows large (Section 4.4).

insignificant at large n. From this we observe that choosing larger cluster size pays off for large graphs, where $s = 50$ gives about 7% less area at 100,000 vertices than $s = 30$. Beyond $s = 50$ the clusters pass the limit of PSSA itself (Section 2.5), which makes $s = 50$ the best choice for large graphs right now. It may be worth extending PSSA wBC to rectangular patches in future work as these can have a larger perimeter to node ratio, and thus can allow more border nodes, whilst retaining the same cluster size s, pushing us further down the T/n curve and achieving a more compact embedding.

At 100,000 vertices, embedding the clusters took under three hours on the 8 cores of my laptop, while the partitioning took under a minute and the closed-form routing at the end took a couple of seconds. It's also worth noting that the only step that has to handle the whole graph at once is the partitioning, which Mt-KaHyPar has done for hypergraphs of up to 140 million vertices (Gottesbüren et al. 2022) and GGP + T-FM for ten million vertices (Section 4.3.2), so nothing but the number of cores stands in the way of far larger graphs. My work has mainly covered random cubic graphs and Erdős–Rényi graphs. Since the area tends to $(T/n)^2$, the fallback beats the complete graph embedding at large n on any graph family whose T/n stays clearly below 1. How T/n behaves on other graph families, and so how well the method does on them, I have not yet studied properly and I leave it to further research (Section 6).

5 Discussion

5.1 Divide and Conquer vs the fallback method

Divide and Conquer and the fallback method share the same philosophy and differ only in how they route the cut edges, so the natural question is their application. Problems of interest in probabilistic computing are large, as the million p-bit computer of Aadit et al. (2026) already runs networks of one million p-bits on FPGAs and a single chip with a million p-bits is projected to be feasible (Nikhar et al. 2024). Clearly, a graph with 1,000 vertices is still toy size next to these, and real scalability means reaching hundreds of thousands to millions of vertices. The fallback method reliably achieves this, as it is guaranteed and runs in linear time, and on my laptop it embedded graphs up to a million vertices. Divide and Conquer is not there yet, as in my runs it embedded no graph beyond 1,400 vertices, so as of now it is best seen as a heuristic method, fitting with the notorious NP-hard scaling of these heuristics. With this identification the Divide and Conquer method can be judged by how much it improves on standalone heuristics such as PSSA (Section 5.2), and for future research, I would like to see how some manifestation of the method could replace PSSA wBC in the fallback method to achieve bigger clusters for the sake of reaching a lower T/n and perhaps decreasing the embedding size. Otherwise, on its own I see that the potential of the Divide and Conquer method at scale lies with networks with community structure and locality, which Section 6.3 also returns to.

At toy size, Divide and Conquer is the more compact method. Figure 21 draws a random cubic graph with 1,000 vertices embedded three ways at the same scale, where the complete graph embedding needs $KG_{999,999}$, the fallback method $KG_{682,682}$, about 2.1 times less area, and Divide and Conquer $KG_{471,471}$, about 4.5 times less than the best known complete graph embedding. To see how Divide and Conquer holds as n grows, I ran both methods on the same random cubic graphs, from 100 vertices to a million with the complete graph embedding as the control (Fig. 22). In my runs, Divide and Conquer was the more compact method wherever it embedded, yet it failed to embed any graph beyond 1,400 vertices, whereas the fallback method needed at most about half the area of the complete graph embedding for every instance, down to 2.76 times less at a million vertices, therefore only the fallback method remains at scale. The two methods greatly differ in tractability. The routing of the fallback method is a closed form that always succeeds, in linear time, while that of Divide and Conquer is an NP-hard search whose chance of success fell as the graphs grew (Section 3.3), as is the nature of heuristic algorithms.

5.2 The treewidth law and the constant coefficient

By the treewidth law of Section 3.3, every method needs an L that grows linearly with n on expanders, as that is the optimal minor-embedding size for this family, so the methods outlined in this report are all order optimal and can differ in their constant coefficients and how those coefficients change as n grows. The constant of Divide and Conquer grew with n, from 9% of the area of the complete graph embedding at 100 vertices to 29% at 1,000 (Fig. 22), as its L follows the cut and some of its routes must cross the whole chip. In contrast, the constant coefficient of the fallback method fell as n grew. It predictably fell towards the floor $(T/n)^2$ as my size formula $L = T + c\sqrt{n/s}$ of Section 4.2 predicts.

To compare with the PSSA heuristic of Sugie et al. (2021), the best known heuristic on random cubic graphs for the King's graph topology, in my runs, my Divide and Conquer method matched it at 100 vertices and later fell behind with size (Fig. 22). It is important to note however, that this is all with my much weaker PSSA, which embeds only about 50 vertices for random cubic graphs in comparison to Sugie's demonstrated 1,000 (Section 2.5), and that as a heuristic, Divide and Conquer carried my PSSA from about 50 vertices to 1,400 while keeping the embeddings compact. Sugie's PSSA is also more compact than the fallback method wherever it has been demonstrated, up to 1,000 vertices, but at scale, where only a guaranteed method will do, the fallback method gives the most compact embedding known.

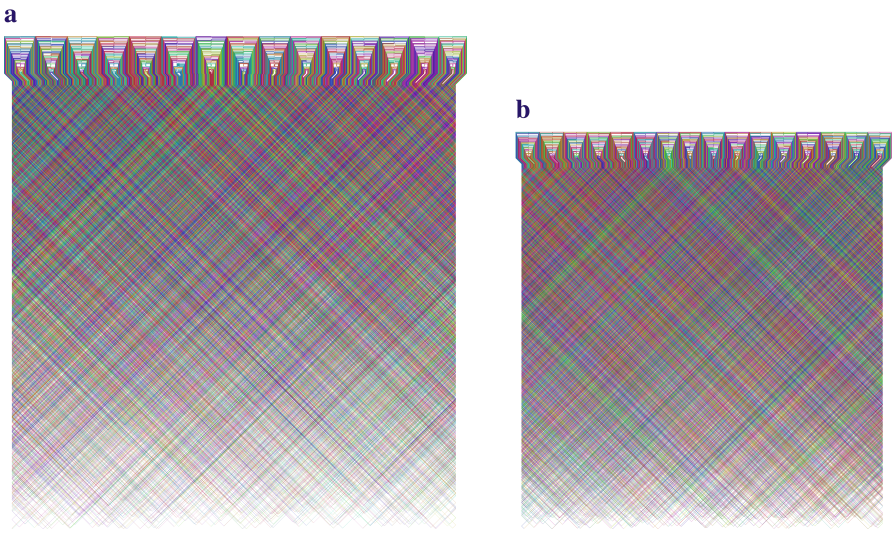


Figure 20 | Two graphs with 10,000 vertices embedded by the fallback method at $s = 50$, drawn to the same scale. **a**, A random cubic graph, on $KG_{6401,6401}$, 2.44 times less area than the complete graph embedding. **b**, An Erdős–Rényi graph generated with mean degree 3, on $KG_{5145,5145}$, 3.8 times less area.

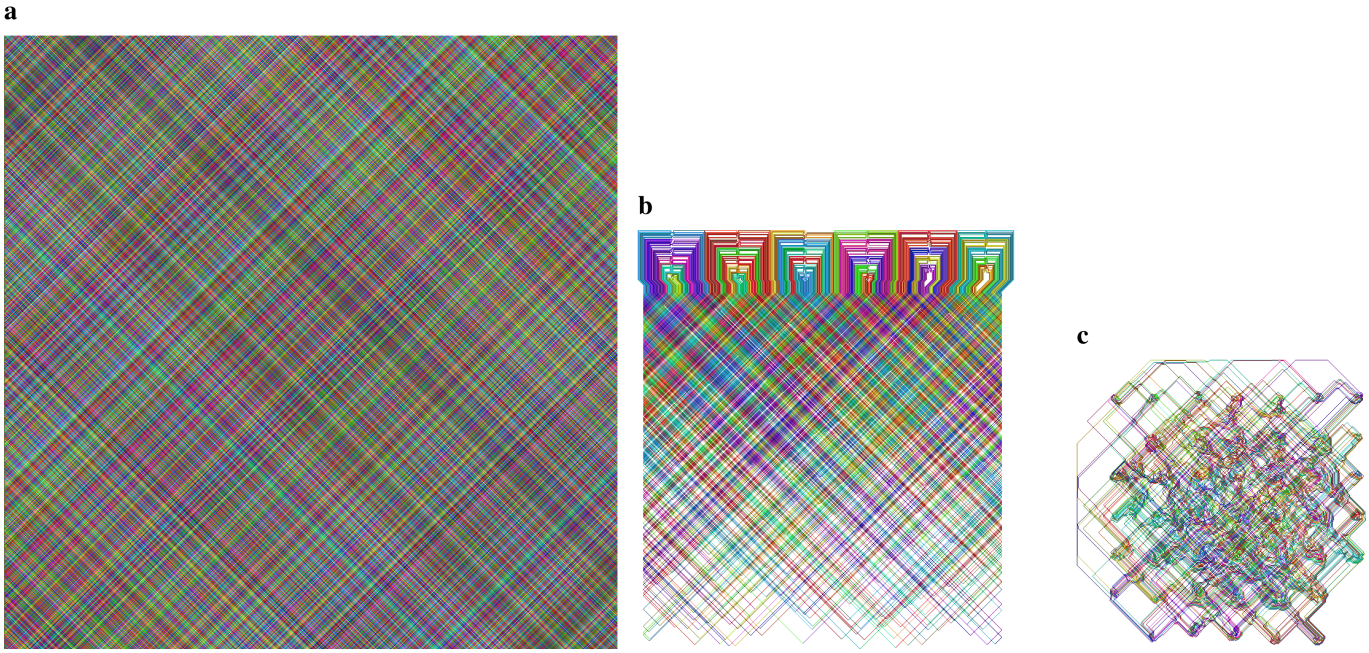


Figure 21 | A random cubic graph with 1,000 vertices embedded by the three methods, drawn to the same scale, with every hardware vertex in the colour of its chain. **a**, The complete graph embedding, on $KG_{999,999}$. **b**, The fallback method at $s = 30$, on $KG_{682,682}$, about 2.1 times less area. **c**, Divide and Conquer, on $KG_{471,471}$, about 4.5 times less area.

5.3 Where each method is at its best

In the terms of Table 1, the two methods are at their best on different families. In Divide and Conquer the overhead in hardware vertices stays small if the cut is small and the connected clusters are placed in proximity keeping the routes short. In my runs on random cubic graphs, the cut grew linearly with n with 485 edges at 1,000 vertices, and the hardware vertices per input vertex grew with it, from about 4 at 100 vertices to 88 at 1,000, as the area of the embedding, L^2, grows quadratically with the cut size (Section 3.3). Planted communities joined by random edges, as in the stochastic block model of Section 3.3, keep the cut small but still give long routes, so the condition is communities that exhibit locality in their connectivity, which perhaps points to the sparse structured bipartite family (Section 6.3).

The fallback method is at its best at scale. Its area against the complete graph embedding tends to $(T/n)^2$, so on expanders it wins by a factor that grows with n, 2.76 times on a random cubic graph with one million vertices and 4.43 times on an Erdős–Rényi graph with one million vertices (Section 4.4). Which graph families it's best on, however, I can't yet say, as partitioning to minimize the border node count is still highly unstudied and personally, I have measured it only on the two families in this report. By the size formula of the fallback method, it does best wherever a graph can be partitioned with few border nodes, ie. a low T/n, and finding these families is a part of further research of Section 6.1. On dense graphs, where T approaches n, the fallback method has nothing to gain over the provably near optimal best known complete graph embedding.

5.4 What limits each method

I find that the two minor-embedding methods are limited by different things. Divide and Conquer firstly employs multiway cut partitioning, which is NP-hard itself, however it has been thoroughly studied and developed over decades and so I don't think that's currently the biggest direction to push on. Thus, excluding partitioning, I identify that the Divide and Conquer

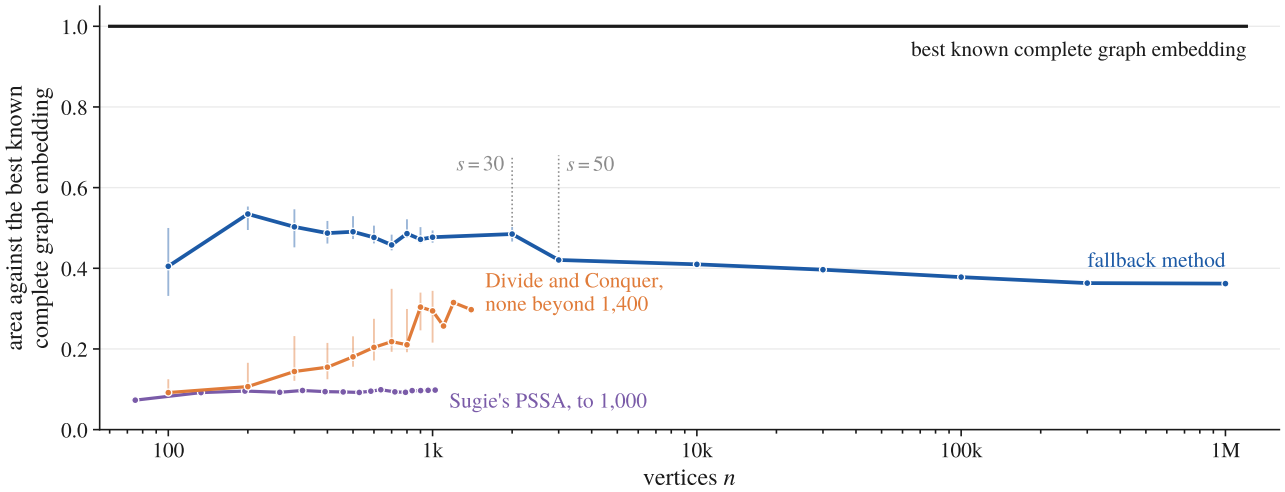


Figure 22 | The four methods on random cubic graphs, with the area of each embedding against the best known complete graph embedding. The points of the improved PSSA of Sugie et al. (2021), the best known heuristic, are the embedding thresholds read off their Fig. 6a, ie. the size at which it stopped embedding 19 of 20 graphs, on hardware up to $KG_{320,320}$, the largest it was demonstrated on. Divide and Conquer is drawn as the medians of ten graphs up to 1,000 vertices, of those it embedded, with bars for their range, continued with single graphs up to 1,400 vertices, the largest it embedded, and the fallback method at the better cluster size for each n, $s = 30$ up to 2,000 vertices, also as the medians of ten graphs, and $s = 50$ from 3,000 with single graphs, up to a million vertices, with the switch between the two marked by dotted lines.

method is limited by two searches; the embedding of its clusters by PSSA wBC and the routing of its cut edges. Both of these processes are NP-hard and intractable for large scale application, and to push Divide and Conquer further one could choose to either push for larger clusters by improving PSSA wBC or to improve the cut edge routing process. In my project, once I implemented my small improvements on PSSA, I accepted the 30–50 vertex limit and looked how far I could push the routing process, and the cut edge routing failed ever more often as the cut grew (Section 3.3). Nevertheless, larger clusters would ease the routing, as they give a smaller cut, so a stronger PSSA wBC would carry Divide and Conquer further (Section 6.3). The fallback method has no search in its routing and it is instead limited by its partition, ie. by how low T/n can fall to minimize the routing structure. Also, I think that being able to embed larger clusters from a stronger PSSA or from Divide and Conquer with border constraints (Section 6.2) could give us more range to access the node distributions with lower T/n on graphs with community structure, by simply easing the cluster size constraint on the multiway cut search. And even on random cubic graphs, T/n keeps falling slowly with s (Section 4.4), so on expanders the limit of the fallback method lies both in the border node minimization partitioning itself and in the size of the clusters that can be embedded, which Sections 6.1 and 6.2 return to.

This report measures the embeddings by their area alone from the perspective of reducing wasted hardware space and real scalability of such hardware, but in practice, how well an embedded network simulates also depends on the lengths of its ferromagnetic chains. Long ferromagnetic chains suffer from chain stiffness, as with couplings too weak the spins of one chain disagree, and with couplings too strong the sampling freezes, and setting their coupling strength is the parameter setting problem of Choi (2008) (Section 1.2). Clustering algorithms, which update a whole chain at once, can help with this. In the fallback method this clustering could be simple, as the long chains are those of the routing structure which is closed form and predictable, so only its chains really have to be clustered, and their shapes and locations are known in advance.

6 Further research

Below are some promising future directions I can think of for future research, starting with the fallback method, and each grows out of a result of this report. Smaller ideas are noted in the sections in which they arise, in the partitioning (Sections 4.3.2 and 4.4) and in the clustering of chains (Section 5.4).

6.1 The fallback method as a general sparse embedder

The fallback method is the leading result of this project and its biggest direction. It is guaranteed, runs in linear time and embeds any graph, and on the families I tested it gave the most compact embedding known at scale, with 2.76 times less area than the best known complete graph embedding on a random cubic graph with a million vertices and 4.43 times less on an Erdős–Rényi graph of the same size (Section 4.4). Minor-embedding sparse networks at scale is a problem shared by probabilistic computing, quantum annealing and Ising machines in general, and I believe the method could contribute to all three, while the observed plateau of T/n for different graph families could perhaps find a place in graph theory itself.

Critically, the fallback method turns the question of how compactly a graph can be embedded into a question of partitioning. By its size formula, as n grows large, L tends to T and the area tends to $(T/n)^2$ (Section 4.2), so everything comes down to the achieved T/n in the partition, and the lowest T/n a graph allows at a given cluster size depends on the properties of the graph itself. The way I see it, the application and further development of this method is largely a question of graph theory, and it's a very open one. On random cubic graphs at $s = 50$, T/n plateaus at about 0.6, which is low in comparison to the 0.8 that minimizing the cut gives (Section 4.3), yet as for the cause and nature of the plateau and how it scales with n and s, I couldn't

find this anywhere. I think the nearest goal is investigating the partitioning method further, as it's a huge area to expand into.

The performance of the fallback method on other graph families is another exciting open question. The method beat the complete graph embedding on both families I tested, and since expanders are seemingly the hardest case among sparse graphs as they lack community structure and their optimal L grows linearly with n which is the most any graph can need (Section 3.3), I see this as a promising sign that it could perform even better on many other graph families, which perhaps hold the particular graph properties that result in a low T/n plateau, and in general, that the method could become a general embedder for sparse graphs. Furthermore, an easy next approach is to empirically search for such low T/n families, starting with the open library of 24,016 graphs of Macaskill-Smith et al. (2026).

6.2 Larger clusters with Divide and Conquer

Besides finding the families with a low T/n, easing the cluster size constraint on the partitioning could lower T/n itself. The fallback method takes its clusters from PSSA wBC, which with my PSSA reaches about 50 vertices, while Divide and Conquer embedded every random cubic graph of up to 700 vertices, nearly always at the first try, whilst climbing up to 1,400 vertices on the tested instances (Section 3.3). In place of PSSA wBC, Divide and Conquer with border constraints could greatly increase the cluster size threshold, and I found that the partition would reward it, as T/n keeps falling with s well past the limit of PSSA, from 0.60 at $s = 50$ to 0.54 at $s = 1{,}000$ on a random cubic graph with 100,000 vertices (Section 4.4). By the size formula, clusters of this size would lower the limit of the area, $(T/n)^2$, from about 2.8 to beyond 3 times less than the complete graph embedding, and this is on expander graphs, which have no community structure to exploit. On graphs with community structure, where a single cluster could hold a whole community, T/n might fall significantly further. Furthermore, Divide and Conquer on its own scales better on communities that connect locally to keep its cut edge routes short, while inside the fallback method every cut edge is realized reliably through the set routing structure (Section 4.1). In this way, Divide and Conquer would only embed the communities and the fallback method would connect them, which could make sparse structured bipartite networks a natural target for this method (Table 1). The method would also remain guaranteed in general, as any cluster Divide and Conquer fails on can still be embedded by the complete graph embedding (Section 4.1).

6.3 Divide and Conquer at scale

As a heuristic, Divide and Conquer is already a very good one. It carried my PSSA from about 50 vertices to 1,400 while keeping its embeddings compact, and at 1,000 vertices it needed a median of 3.4 times less area than the complete graph embedding, which is substantial compared to the 2.1 times for my fallback method on the same graphs (Section 5.1). It matched the area of the improved PSSA of Sugie et al. (2021), the best known heuristic on the King's graph, at 100 vertices, and needed at most about three times its area up to 1,000 (Fig. 22), all with my far weaker PSSA inside it. With a stronger cluster embedder, such as Sugie's PSSA or minorminer (Cai et al. 2014), it could likely perform and scale much better. At scale, however, it keeps its overhead small only when its cut is small and its routes are short, ie. on networks whose communities connect locally (Section 5.3), so the way I see it, its future as a scalable method lies with local, low-degree networks, such as the local restricted Boltzmann machines of the sparse structured bipartite family (Section 1.5), a family on which the leading work of the field already builds as shown by the Camsari group just recently in June 2026. The group's million p-bit computer partitions its sparse network over 18 FPGAs with a min-cut partitioner and places strongly connected partitions on nearby devices (Aadit et al. 2026), which are similar to the first two steps of my Divide and Conquer method, only on reconfigurable FPGAs, where each piece runs as it is and the links between the devices carry the cut edges, so that no minor-embedding is needed. Chowdhury et al. (2025) likewise map a restricted Boltzmann machine with strictly local connections onto neighbouring FPGAs, and Niazi et al. (2024) show that a sparse deep Boltzmann machine of degree 15 to 20 matches the accuracy of a fully connected restricted Boltzmann machine with about a hundred times fewer parameters. My method brings this same strategy to the King's graph, a regular and scalable hardware topology, and though the Divide and Conquer method is not yet scalable itself it shows that the strategy works on such a chip.

Two things stand between it and these networks. The first is degree, as my PSSA wBC manages a degree of 3 to 4 today (Fig. 12), while these networks have 13 to 20 neighbours per p-bit, which calls for either a stronger PSSA or sparsifying first with COPY gates, which split a p-bit into copies coupled ferromagnetically, at the cost of auxiliary p-bits (Aadit et al. 2022). The second is the routing, which remains a search on an NP-hard problem (Section 3.3). Furthermore, the sparsification approach is noteworthy, as any dense network sparsified with COPY gates to the degree my PSSA handles, then partitioned and embedded by Divide and Conquer or by the fallback method is a feasible method for general embedding (Section 1.2) and could be tackled in the future.

6.4 Running on real hardware

Finally, these improved and scalable embeddings are meant to run and anneal on real hardware, and the next step is to actually simulate embedded spin networks. Following the path my project proposal set out (Sanvito 2026), from simple truth tables to optimization, the logic of a problem would be formalized algebraically in Quadratic Unconstrained Binary Optimization (QUBO) along with a coupling matrix of spins, then this network would be mapped onto hardware by one of my minor-embedding methods, and then it would be annealed and simulated to check that it still finds the ground states of the original network and that the functionality hasn't been corrupted. Annealing quality could depend on chain stiffness as well as general ferromagnetic coupling strengths (Choi 2008) (Section 1.2), and on clustering, which for the fallback method could be simple as only the predictable chains of its routing structure would need clustering (Section 5.4).

The King's graph remains the hardware I see the most promise in due to its feasibility and scalability, but the methods could also be tested on D-Wave's Chimera lattice, on which much of the work on minor-embedding builds (Cai et al. 2014; Sugie et al. 2018). The King's graph is a minor of Chimera, and as a minor of a minor is again a minor, every embedding of this report carries over to Chimera directly. In the other direction, a planarity test, which runs in linear time (Hopcroft and Tarjan

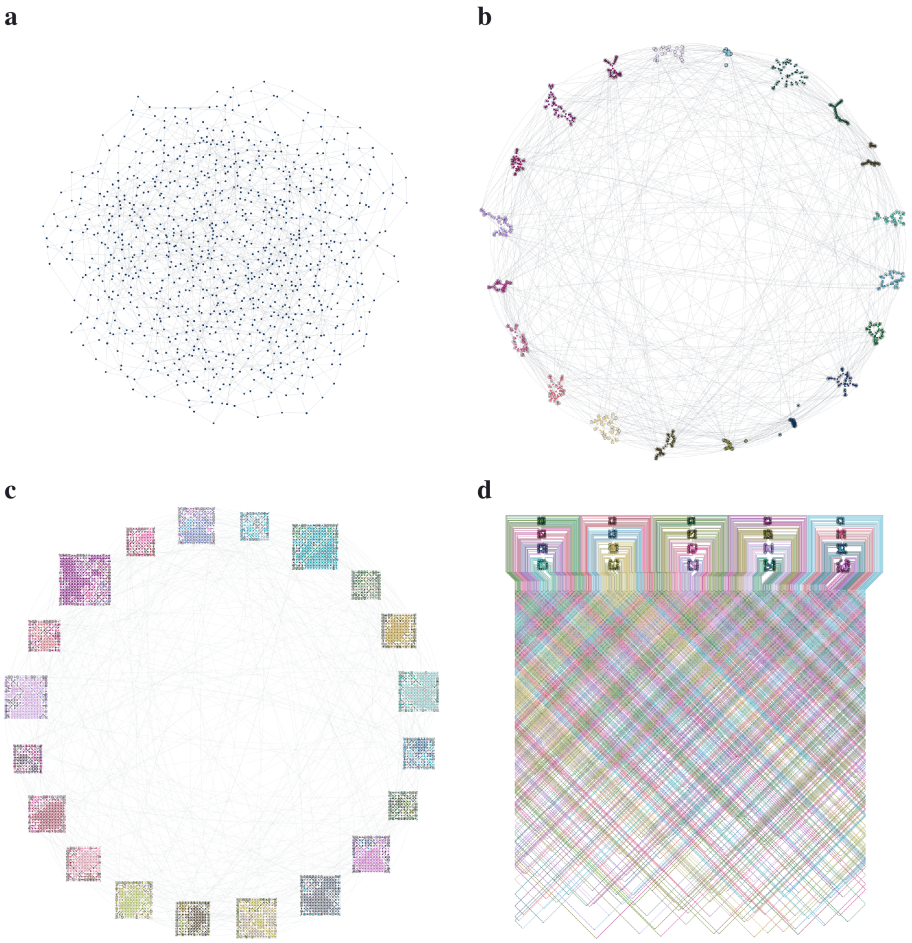


Figure 23 | The fallback method on a random cubic graph with 960 vertices at $s = 50$, replayed from its recorded run, in four frames of the live figure. **a**, The input graph, before partitioning. **b**, Partitioning for the fewest border nodes gives 20 clusters with 529 border nodes and 451 cut edges. **c**, Each cluster is embedded on its own hardware patch by PSSA wBC with its border nodes on the perimeter, here midway through, and during the cluster embedding the live figure's plot follows the geometric annealing schedule and the PSSA cost function E_{emb}. **d**, The clusters are then placed in a grid layout and every border node is uniquely routed to the routing structure, where every cut edge is realized due to its all-to-all structure, giving a complete and valid embedding of all 1,440 edges of the 960 vertex cubic graph on $\text{KG}_{638,638}$, about 2.3 times less area than the complete graph embedding on $\text{KG}_{959,959}$. The whole run plays on the live page, or from the QR code below.

1974), could send every planar network to the simpler square lattice, which hosts all of them (Section 1.3), though this is already known.

7 Conclusion

As of now, large sparse networks have no minor-embedding method that is both compact and guaranteed. For arbitrarily large problems the field relies on the complete graph embedding, which is blind to sparsity and any other graph properties and is extremely wasteful in practice, and in fact the majority of the field sidesteps the problem altogether by compromising for FPGAs. To close this gap, I first had to choose the fixed hardware architecture I wanted, and I noticed that sparsifying a network with COPY gates and minor-embedding it are one and the same operation, therefore no amount of COPY gate sparsification can make a non-planar network planar. From this I identified that non-planarity is a requirement for a truly universal hardware architecture, which led me to the King's graph, the simplest universal hardware architecture as I see it, whose complete graph embedding is optimal to within a single vertex (Sugie et al. 2021). I built two embedding methods on the King's graph hardware architecture, both of which rest on my PSSA with border constraints. PSSA wBC embeds every cluster of a partitioned network on its own patch with its border nodes on the perimeter, so that only the cut edges between the clusters remain to be routed, reducing the large combinatorially hard network embedding problem to a slightly simpler one.

My Divide and Conquer method routes the cut edges by a heuristic search, following the classic strategy of electronic design automation, similar to the partitioning and placing of Aadit et al. (2026) on FPGAs, but on fixed hardware instead where every cut edge must be routed through the fixed King's graph architecture itself. As a new proof of concept it is already quite compact. It carried my PSSA from about 50 vertices to 1,400, and it minor-embedded a random cubic graph with 100 vertices on $\text{KG}_{25,25}$, matching the improved PSSA of Sugie et al. (2021). At 1,000 vertices it needed a median of 3.4 times less area than the complete graph embedding, which is significantly more than the 2.1 times for the fallback method. Its routing, however, is a hard heuristic search, and in my runs it embedded no graph beyond 1,400 vertices. How far the approach itself scales I can't yet say, as it is still highly undeveloped. A stronger cluster embedder, such as Sugie's original PSSA or minorminer (Cai et al. 2014), more intelligent placement of the clusters, and networks that inherently connect locally, such as those of the sparse structured bipartite family, could all carry the method much further. Furthermore, Divide and Conquer itself acts as a heuristic minor-embedding algorithm, just like PSSA, so a method built on PSSA wBC could in turn take its place. With border constraints, Divide and Conquer could supply the clusters of the fallback method, reaching far past the 50 node cluster size cap towards clusters in the hundreds of nodes, which might lower T/n, and with it the area of the embedding.

The fallback method is my second embedding method, and it routes the cut edges through a complete graph embedding of the border nodes alone. It is therefore guaranteed, runs in linear time and scales to large problems, for which the clusters minor-embed in parallel and the routing structure takes only seconds to realize (Fig. 23). Its size follows the equation $L = T + c\sqrt{n/s}$, so at large scale its area tends to $(T/n)^2$ of that of the complete graph embedding. I identified that the partitioning stage should

minimize the number of border nodes T to minimize the area of the embedding, and interestingly I found that this objective is in opposition with the classical objective of minimizing the cut. Partitioning for the fewest border nodes brings T/n to a plateau of about 0.6 on large random cubic graphs at $s = 50$. Results show that the fallback method needs 2.64 times less area than the complete graph embedding on a random cubic graph with 100,000 vertices, and 4.12 times less on an Erdős–Rényi graph of the same size, which are the most compact embeddings known for the graph families at this scale. At a million vertices, the scale of the largest probabilistic computers today (Aadit et al. 2026), it needs 2.76 times less area than the best known complete graph embedding on a random cubic graph and 4.43 times less on an Erdős–Rényi graph. On expanders every method needs a side L that grows linearly with n, so all of the methods are order optimal and differ only in their constant coefficients. The constant coefficient of Divide and Conquer grew with n, while that of the fallback method falls towards $(T/n)^2$, so its gain over the complete graph embedding grows with the size of the network.

Critically, the fallback method turns the minor-embedding problem into a partitioning one, and which network graphs it embeds compactly is now largely a question of graph theory, and a very open one at that. What sets the plateau of T/n, whether a plateau exists on all graph families, how T/n scales with n and the cluster size s, and which graph families attain a low T/n, I can't yet answer, as partitioning to minimize the border nodes is, as far as I could find, still largely unstudied. Furthermore, none of these embeddings has yet been annealed on real hardware, where their long chains bring the parameter setting problem of Choi (2008). Simulating them is the natural first step, to check that the minor-embedded networks converge to the correct ground states and reliably emulate the logic of the problem they're built for. The fallback method already wins significantly on expanders, seemingly the hardest case among sparse graphs, which is a good sign that it could become a general embedder for sparse graphs that is sensitive to the properties of the graphs themselves, unlike any other scalable method in the field. Making it so is what I most want to do next, for on regular, fixed hardware such an embedder could carry the networks of real problems to the scale of billions of p-bits.

Code availability

The code of this project is available at https://github.com/SzymonAblewicz/Topologically-Restricted-P-Computing. It holds the improved PSSA, PSSA wBC and the Divide and Conquer and fallback methods, together with the scripts and data behind the results of this report. The PSSA code is built on that of Ember (Macaskill-Smith et al. 2026), under its MIT licence.

Acknowledgements

I would like to thank Prof. Stefano Sanvito for supervising this project, for our weekly meetings, which guided it throughout, and for welcoming me into his team at CRANN, Trinity College Dublin's nanoscience research institute, for the duration of the summer. I thank him and Michael Mitchell for their advice and help on the work, and Michael for the idea of adding Gibbs weighting to the swap and shift moves, which improved PSSA (Section 2.3.1). I also thank CRANN for funding the project, and the School of Mathematics at Trinity College Dublin for funding six weeks of it through its Hamilton Trust internship, during which I had the chance to present it at a seminar and in a poster session.

References

Aadit, Navid Anjum et al. (2022). "Massively parallel probabilistic computing with sparse Ising machines". In: *Nature Electronics* 5.7, pp. 460–468. DOI: 10.1038/s41928-022-00774-2.

Aadit, Navid Anjum et al. (2026). *Programmable Probabilistic Computer with 1,000,000 p-bits*. arXiv: 2606.25313.

Breuer, Melvin A. (1977). "Min-Cut Placement". In: *Journal of Design Automation and Fault-Tolerant Computing* 1.4, pp. 343–362.

Cai, Jun, William G. Macready, and Aidan Roy (2014). *A practical heuristic for finding graph minors*. arXiv: 1406.2741.

Camsari, Kerem Y., Shuvro Chowdhury, and Supriyo Datta (2019). "Scalable Emulation of Sign-Problem-Free Hamiltonians with Room-Temperature p-bits". In: *Physical Review Applied* 12, p. 034061. DOI: 10.1103/PhysRevApplied.12.034061.

Çatalyürek, Ümit V. and Cevdet Aykanat (1999). "Hypergraph-partitioning-based decomposition for parallel sparse-matrix vector multiplication". In: *IEEE Transactions on Parallel and Distributed Systems* 10.7, pp. 673–693. DOI: 10.1109/71.780863.

Chang, Yun-Chih et al. (2000). "B*-Trees: A New Representation for Non-Slicing Floorplans". In: *Proceedings of the 37th Design Automation Conference*, pp. 458–463. DOI: 10.1145/337292.337541.

Choi, Vicky (2008). "Minor-embedding in adiabatic quantum computation: I. The parameter setting problem". In: *Quantum Information Processing* 7.5, pp. 193–209. DOI: 10.1007/s11128-008-0082-9.

Choi, Vicky (2011). "Minor-embedding in adiabatic quantum computation: II. Minor-universal graph design". In: *Quantum Information Processing* 10.3, pp. 343–353. DOI: 10.1007/s11128-010-0200-3.

Chowdhury, Shuvro et al. (2023). "A Full-Stack View of Probabilistic Computing With p-Bits: Devices, Architectures, and Algorithms". In: *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits* 9.1, pp. 1–11. DOI: 10.1109/JXCDC.2023.3256981.

Chowdhury, Shuvro et al. (2025). *Probabilistic Computers for Neural Quantum States*. arXiv: 2512.24558.

Diestel, Reinhard (2005). *Graph Theory*. 3rd ed. Vol. 173. Graduate Texts in Mathematics. Springer.

Fiduccia, Charles M. and Robert M. Mattheyses (1982). "A linear-time heuristic for improving network partitions". In: *Proceedings of the 19th Design Automation Conference*, pp. 175–181. DOI: 10.1109/DAC.1982.1585498.

Gottesbüren, Lars et al. (2022). "Shared-Memory n-level Hypergraph Partitioning". In: *Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX)*, pp. 131–144. DOI: 10.1137/1.9781611977042.11.

Grohe, Martin and Dániel Marx (2009). "On tree width, bramble size, and expansion". In: *Journal of Combinatorial Theory, Series B* 99.1, pp. 218–228. DOI: 10.1016/j.jctb.2008.06.004.

Hopcroft, John and Robert Tarjan (1974). "Efficient Planarity Testing". In: *Journal of the ACM* 21.4, pp. 549–568. DOI: 10.1145/321850.321852.

Hung, Jui-Hsin and Jaijeet Roychowdhury (2024). *A Study on Advanced Ising Machine Designs: BLIM, King's Graph Mapping, and OIM Chips*. Tech. rep. UCB/EECS-2024-95. EECS Department, University of California, Berkeley. URL: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2024/EECS-2024-95.html.

Kaiser, Jan and Supriyo Datta (2021). "Probabilistic computing with p-bits". In: *Applied Physics Letters* 119, p. 150503. DOI: 10.1063/5.0067927.

Karypis, George and Vipin Kumar (1998). "A fast and high quality multilevel scheme for partitioning irregular graphs". In: *SIAM*

Journal on Scientific Computing 20.1, pp. 359–392. DOI: 10.1137/S1064827595287997.

Kernighan, Brian W. and Shen Lin (1970). "An efficient heuristic procedure for partitioning graphs". In: *The Bell System Technical Journal* 49.2, pp. 291–307. DOI: 10.1002/j.1538-7305.1970.tb01770.x.

Macaskill-Smith, Zachary et al. (2026). *Ember: An Extensible Benchmark Suite for Quantum Annealing Embedding Algorithms*. arXiv: 2604.25433. URL: https://github.com/zachmacsmith/ember.

McMurchie, Larry and Carl Ebeling (1995). "PathFinder: A Negotiation-Based Performance-Driven Router for FPGAs". In: *Proceedings of the ACM Third International Symposium on Field-Programmable Gate Arrays*, pp. 111–117. DOI: 10.1109/FPGA.1995.242049.

Ng, Andrew Y., Michael I. Jordan, and Yair Weiss (2002). "On Spectral Clustering: Analysis and an Algorithm". In: *Advances in Neural Information Processing Systems 14*, pp. 849–856.

Niazi, Shaila et al. (2024). "Training deep Boltzmann networks with sparse Ising machines". In: *Nature Electronics* 7.7, pp. 610–619. DOI: 10.1038/s41928-024-01182-4.

Nikhar, Srijan et al. (2024). "All-to-all reconfigurability with sparse and higher-order Ising machines". In: *Nature Communications* 15, p. 8977. DOI: 10.1038/s41467-024-53270-w.

Ohno, Kentaro and Nozomu Togawa (2024). *Linearizing Binary Optimization Problems Using Variable Posets for Ising Machines*. arXiv: 2307.05125.

Okuyama, Takuya et al. (2016). "Contractive graph-minor embedding for CMOS Ising computer". In: *IEICE Technical Report* 116, pp. 97–103.

Robertson, Neil, Paul Seymour, and Robin Thomas (1994). "Quickly Excluding a Planar Graph". In: *Journal of Combinatorial Theory, Series B* 62.2, pp. 323–348. DOI: 10.1006/jctb.1994.1073.

Sanvito, Stefano (2026). *Topologically Restricted p-Computing*. Hamilton Trust project proposal.

Sugie, Yuya et al. (2018). "Graph Minors from Simulated Annealing for Annealing Machines with Sparse Connectivity". In: *Theory and Practice of Natural Computing (TPNC 2018)*. Ed. by David Fagan et al. Vol. 11324. Lecture Notes in Computer Science. Cham: Springer, pp. 111–123. DOI: 10.1007/978-3-030-04070-3_9.

Sugie, Yuya et al. (2021). "Minor-embedding heuristics for large-scale annealing processors with sparse hardware graphs of up to 102,400 nodes". In: *Soft Computing* 25, pp. 1731–1749. DOI: 10.1007/s00500-020-05502-6.

Tamassia, Roberto (1987). "On Embedding a Graph in the Grid with the Minimum Number of Bends". In: *SIAM Journal on Computing* 16.3, pp. 421–444. DOI: 10.1137/0216030.

Ye, Yinyu (2003). *Gset: A Benchmark Library for Graph Partitioning and Max-Cut*. URL: https://web.stanford.edu/~yyye/yyye/Gset/.

Zaribafiyan, Arman, Dominic J. J. Marchand, and Seyed Saeed Changiz Rezaei (2017). "Systematic and deterministic graph minor embedding for Cartesian products of graphs". In: *Quantum Information Processing* 16.5, p. 136. DOI: 10.1007/s11128-017-1569-z.